



**EKOPARTY
HACKADEMY**

DOCENTE : Florencia Priscilla Vela



Hola!

SOY FLOR

Soy ingeniera en informática y me especializo en inteligencia artificial.

Mail: `flor.p.vela@gmail.com`

Github: `@florvela`

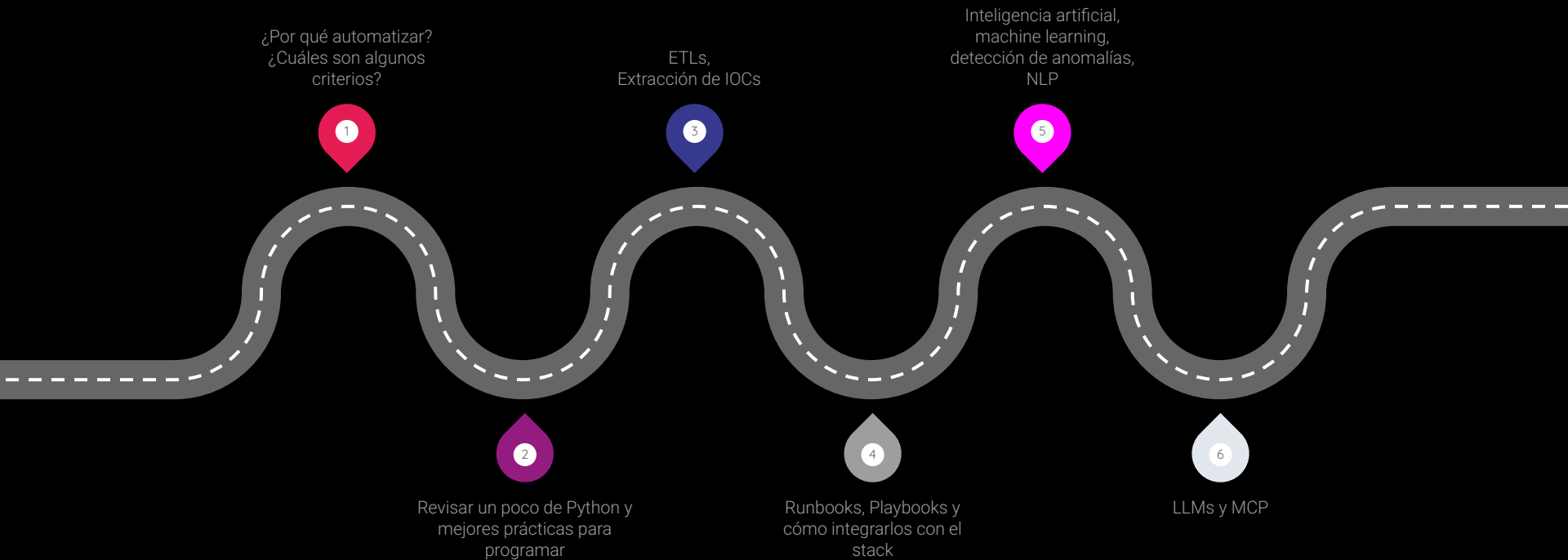
Repositorio de la materia:

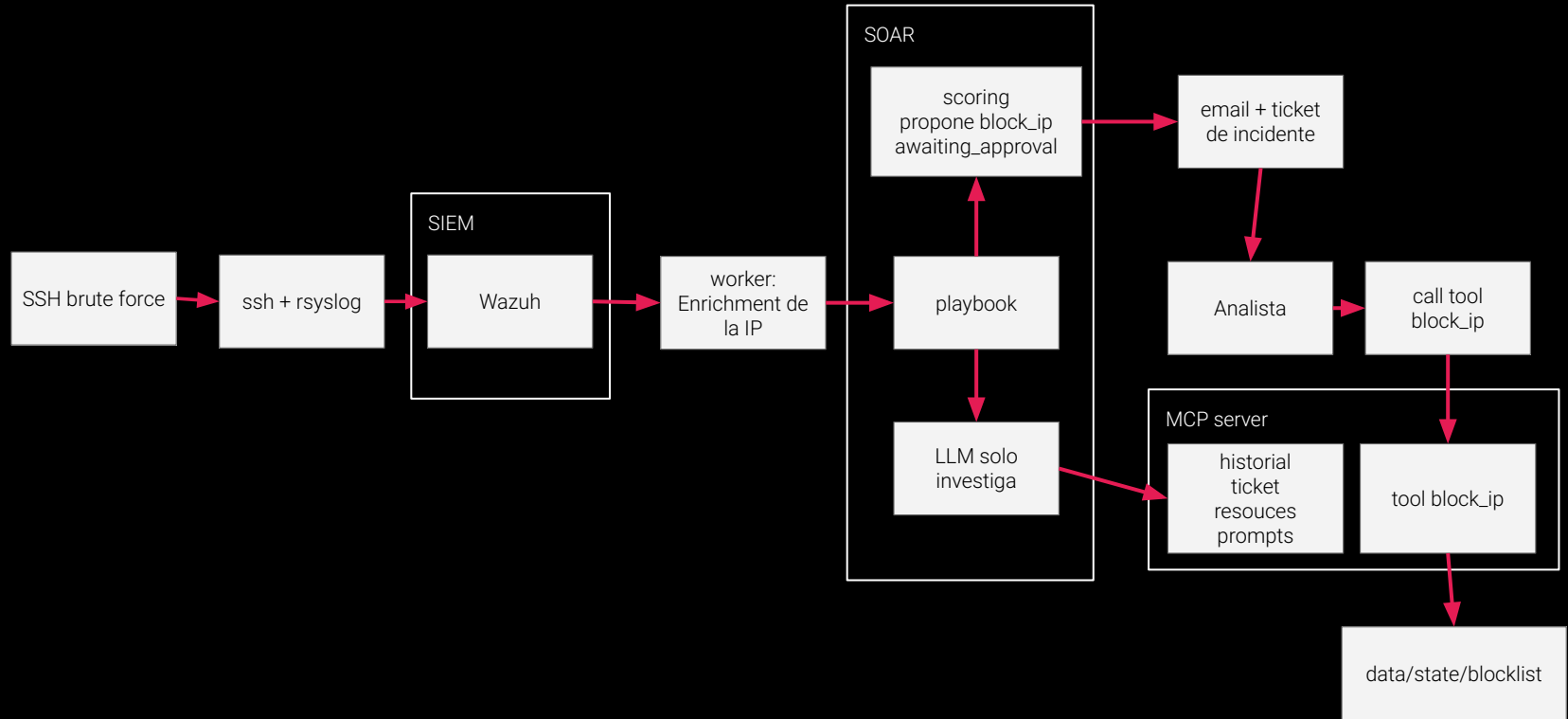
<https://github.com/florvela/IA-y-automatizacion-en-seguridad-defensiva>

ANTES DE EMPEZAR

“No hay que aprender todo de memoria...
Hay que saber qué googlear”

ROADMAP





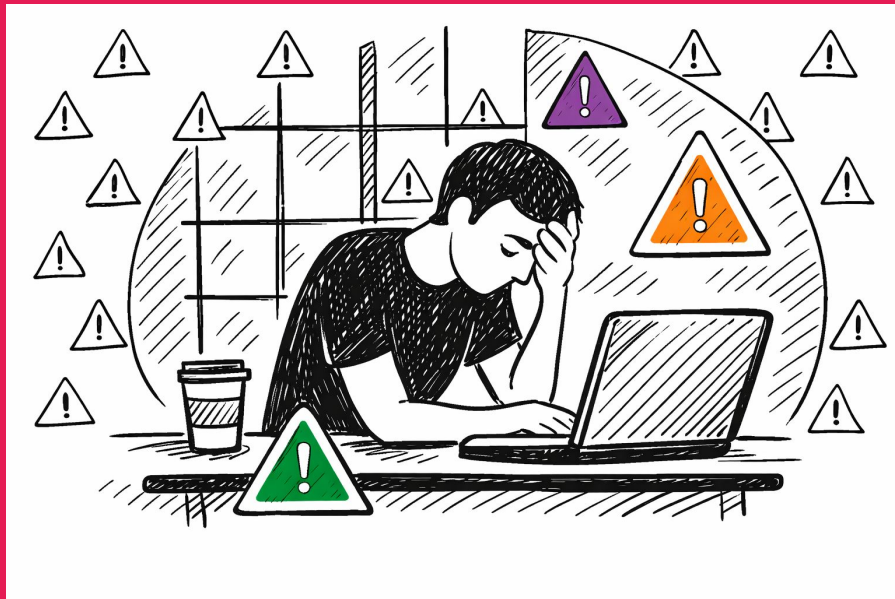


¿POR QUÉ AUTOMATIZAR?

1.000 a

10.000

alertas por día



¿Por qué tantas alertas?

- Reglas amplias que capturan ruido
- Falta de contextualización
- Múltiples fuentes repitiendo eventos
- Reglas obsoletas de un entorno antiguo



Falsos positivos

(falso positivo)



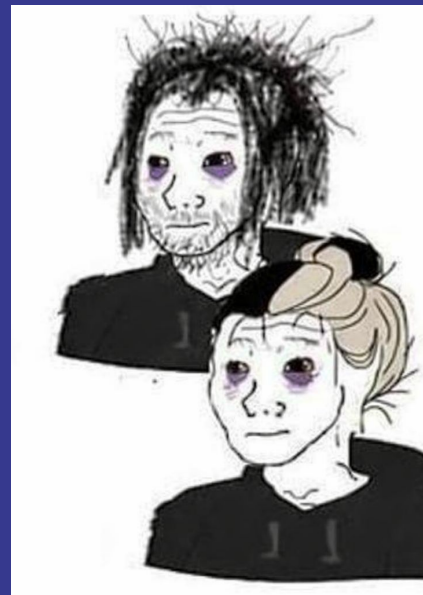
(falso negativo)



Fatiga del analista

Agotamiento cognitivo por
revisar miles de alertas
repetitivas sin impacto real.

- Incidentes reales pasan desapercibidos
- Se extiende el dwell time
- Baja calidad de investigaciones



Por qué automatizar es
OBLIGATORIO

Limitaciones del modelo manual

Velocidad de respuesta



Costo operacional



Tasa de error humano



Beneficios de la automatización en el SOC

Velocidad: Respuesta instantánea

Escalabilidad: Menos costo
por más eventos

Consistencia: Procesos sin errores

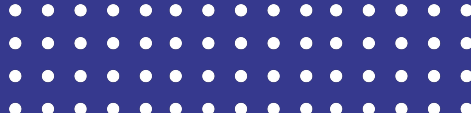
Alivio para los analistas:

Menos tareas aburridas



Cuando **NO** automatizar

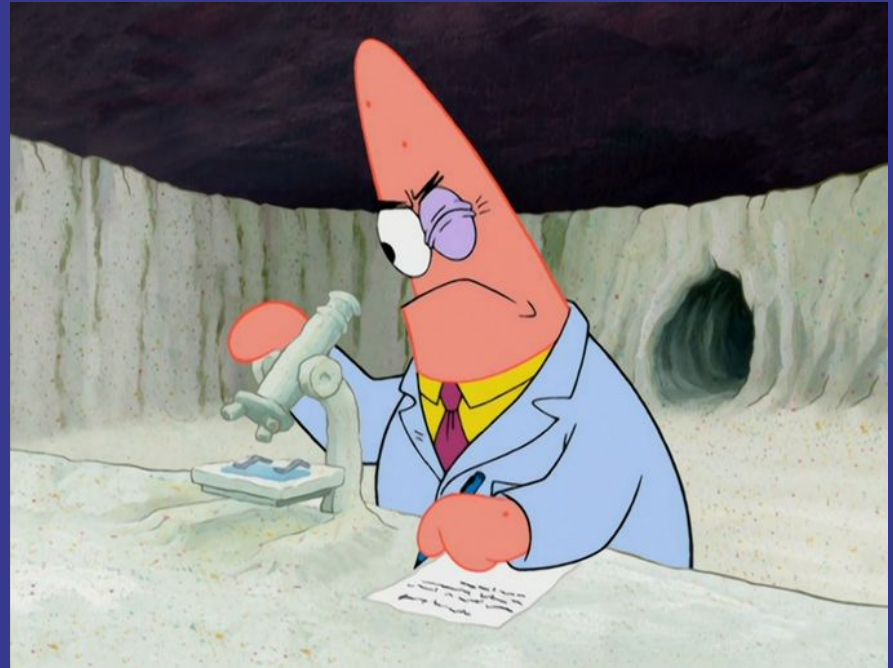
La automatización no siempre es la respuesta





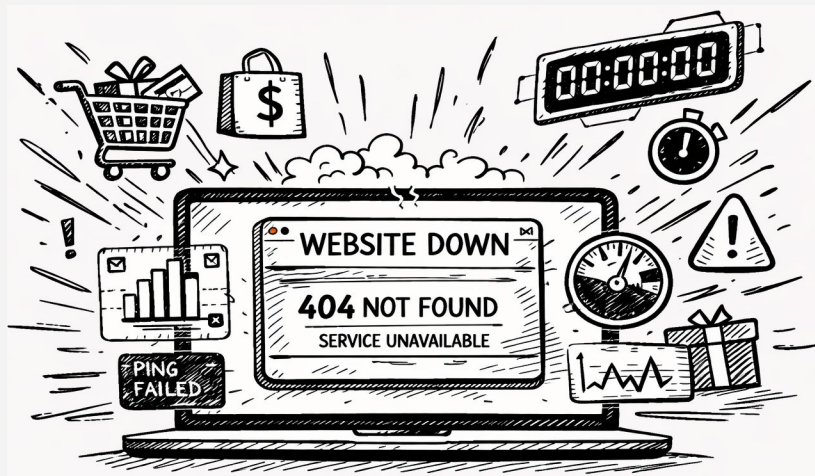
Decisiones que requieren juicio humano

- si tienen muchas variables subjetivas
- si afectan empleados
- si no ocurren frecuentemente
- si no hay precedentes



Riesgo de falsos positivos en acciones bloqueantes

Supongamos que tenemos un pico de tráfico y lo confundimos con un DDoS y bajamos el servidor de nuestro sitio web de ventas... pero es "Black Friday"



Escenarios de alto impacto y baja tolerancia al error

Servidores importantes

Si los aislás por error, podés frenar sistemas clave del negocio.

Red y sistemas centrales

Si bloqueás algo importante, puede caerse toda la operación.

Backups y acceso físico

Un error puede hacer perder datos o dejar gente sin acceso a la oficina.

Human-in-the-loop

Automatización completa:



Human-in-the-loop:

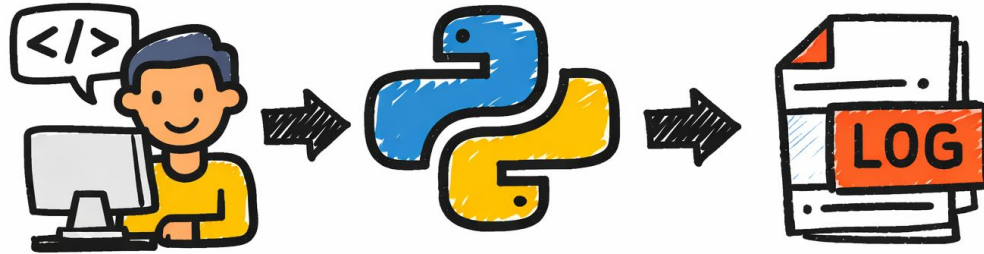


Estado	Acción propuesta
awaiting_approval	block_ip:172.20.0.66 Aprobar bloqueo Rechazar



“

*No se trata de automatizar todo, sino
de automatizar lo correcto.*



¿Por qué Python?



Simplicidad y legibilidad

En comparación con otros lenguajes, necesita menos código para hacer lo mismo,.



Velocidad de desarrollo

Permite trabajar con rapidez: crear scripts, probar ideas y hacer ajustes sin tanta complejidad.



Ecosistema de librerías

Ya existen herramientas listas para usar para conectarse a APIs, procesar datos, trabajar con servidores, cifrado, etc.



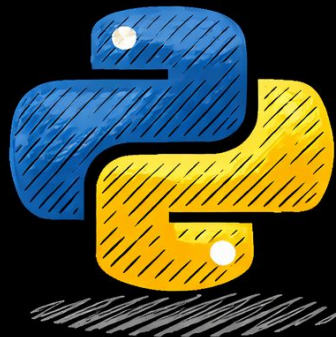
Integración seamless

Se integra bien con muchas plataformas y herramientas que ya se usan en ciberseguridad.



Talento disponible

Muchos analistas ya tienen experiencia con Python. Scripts más fáciles de mantener y entender por otros en el equipo.



Librerías esenciales para seguridad

requests

Consumir cualquier API. VirusTotal, Crowdstrike, Splunk... todo con 2 líneas.

```
import requests
resp = requests.get(url, headers={"x-apikey": KEY})
data = resp.json()
```

paramiko

SSH automático. Ejecutá comandos en servidores remotos para contención

```
ssh.exec_command("ps aux | grep malware")
```

hashlib

Calcular hashes criptográficos (MD5, SHA256). Huellas digitales de archivos.

```
import hashlib
sha256 = hashlib.sha256(bytes).hexdigest()
```

datetime

Manipular timestamps. Parsear formatos distintos. calcular duraciones.

```
from datetime import datetime, timedelta
hace_24h = datetime.now() - timedelta(days=1)
```

Librerías esenciales para seguridad

requests

Consumir cualquier API. VirusTotal, Crowdstrike, Splunk... todo con 2 líneas.

```
import requests
resp = requests.get(url, headers={"x-apikey": KEY})
data = resp.json()
```

```
import requests
```

```
r = requests.post(SOAR_URL, json=alert, timeout=10)
print(f"[siem-lite] ALERTA enviada al SOAR ({ip}) -> HTTP {r.status_code}: {r.text[:120]}")
```

Librerías esenciales para seguridad

paramiko

SSH automático. Ejecutá comandos en servidores remotos.

```
services > attacker >  brute_force.py
```

```
def try_login(user: str, password: str) -> bool | None:
    """Un intento de login. True=éxito, False=credencial inválida, None=error de red."""
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    try:
        client.connect(
            TARGET_HOST,
            port=TARGET_PORT,
            username=user,
            password=password,
            timeout=5,
            allow_agent=False,
            look_for_keys=False,
        )
        return True
    except paramiko.AuthenticationException:
        return False
```

Librerías esenciales para seguridad

datetime

Manipular timestamps. Parsear formatos distintos. calcular duraciones.

```
from datetime import datetime, timezone

def create_incident_ticket(title: str, severity: str, summary: str) -> str:
    """Crea un ticket de incidente y lo guarda. Devuelve el ticket_id."""
    tickets_dir = STATE_DIR / "tickets"
    tickets_dir.mkdir(parents=True, exist_ok=True)
    ticket_id = f"INC-{uuid.uuid4().hex[:8].upper()}"
    ticket = {
        "ticket_id": ticket_id,
        "title": title,
        "severity": severity,
        "summary": summary,
        "created_at": datetime.now(timezone.utc).isoformat(),
        "status": "open",
    }
    (tickets_dir / f"{ticket_id}.json").write_text(json.dumps(ticket, indent=2, ensure_ascii=False))
    return json.dumps({"ticket_id": ticket_id, "status": "created"})
```


BUENAS PRÁCTICAS **para scripts de automatización**

Credenciales - modularidad - logging - testing - resiliencia



GESTIÓN SEGURA DE CREDENCIALES



Usa variables de entorno



Utiliza un gestor de secretos



Escanea antes de hacer commit



Codificar credenciales en el código



Enviar archivos .env



Guardar secretos en el código



NUNCA hagas esto

```
api_key = "sk_live_1234567890abcdefghijklmnopqrstuvwxyz"
password = "SuperSecure123!@#"
# Cualquiera acceso al repo = credenciales expuestas
```



Variables de entorno

```
import os
api_key = os.getenv("VT_API_KEY")
if not api_key:
    raise ValueError("No configurada")
```



Archivos .env

```
# .env (NUNCA en git)
VIRUSTOTAL_API_KEY=sk_live_...
SPLUNK_HOST=splunk.company.com

from dotenv import load_dotenv
load_dotenv()
```

v LABORATORIO-SOC-IA

- > data
- > notebooks
- > services
- > shared
- > soc
- > tests
- > wazuh
- ⚙ .env
- ≡ .env.example
- 🔗 .gitignore
- ⬇ diagrama.md
- 👉 docker-compose.yml
- ⬇ GUIÓN-CLASE.md
- M Makefile
- 📖 README.md
- ≡ requirements.txt

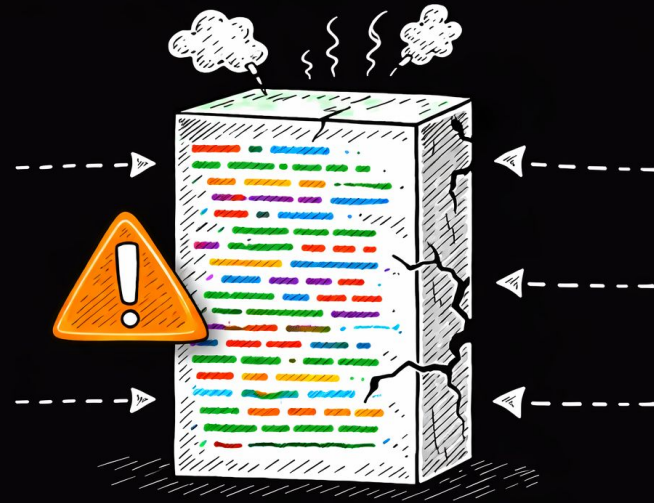
≡ .env.example

```
1  # Copiá este archivo a .env y ajustá lo que necesites.
2
3  # Backend del LLM para la investigación:
4  #   mock    -> determinístico y RÁPIDO
5  #   ollama  -> modelo local llama3.2 (si tarda >45s, cae a mock solo)
6  #   openai  -> API compatible OpenAI (requiere SOC_OPENAI_API_KEY)
7  SOC_LLM_BACKEND=mock
8
9  # Si usás ollama:
10 SOC_OLLAMA_MODEL=llama3.2
11
12 # Si usás openai (o compatible):
13 SOC_OPENAI_API_KEY=
14 SOC_OPENAI_MODEL=gpt-4o-mini
15
16 # ¿El atacante ataca solo al arrancar? (true/false)
17 AUTO_ATTACK=false
18
19 # URL pública del dashboard (links en emails)
20 SOC_DASHBOARD_URL=http://localhost:9000
21
22 # — Email real (opcional) —
23 # Vacío = solo guarda .eml en /data/state/mailbox (modo demo).
24 # Gmail: activá 2FA y creá una "App password" en
25 #   https://myaccount.google.com/apppasswords
26 # SOC_SMTP_HOST=smtp.gmail.com
27 # SOC_SMTP_PORT=587
28 # SOC_SMTP_STARTTLS=true
29 # SOC_SMTP_USER=tu.mail@gmail.com
30 # SOC_SMTP_PASSWORD=xxxx xxxx xxxx xxxx
31 # SOC_EMAIL_FROM=tu.mail@gmail.com
32 # SOC_EMAIL_TO=tu.mail@gmail.com
```

MODULARIDAD

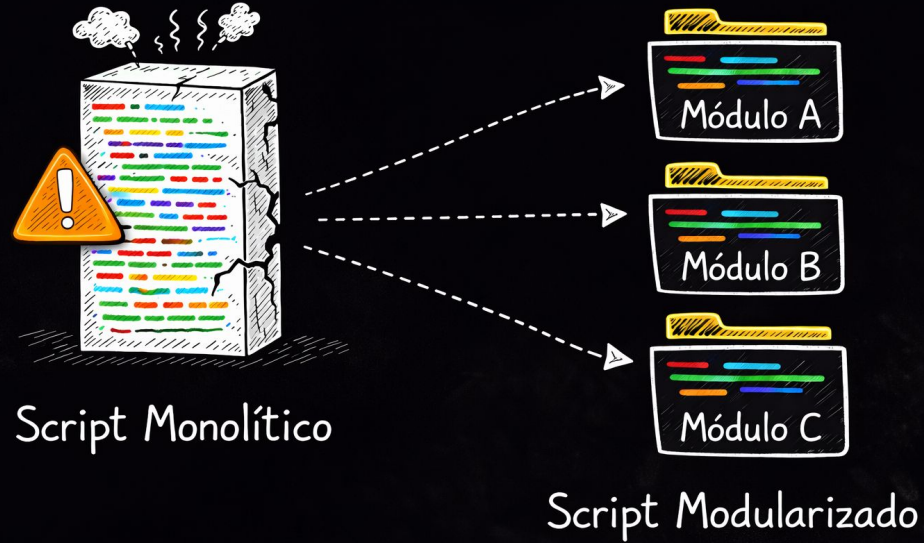
Scripts monolíticos

- Todo en un solo archivo
- Difícil de testear, reutilizar y debuggear.

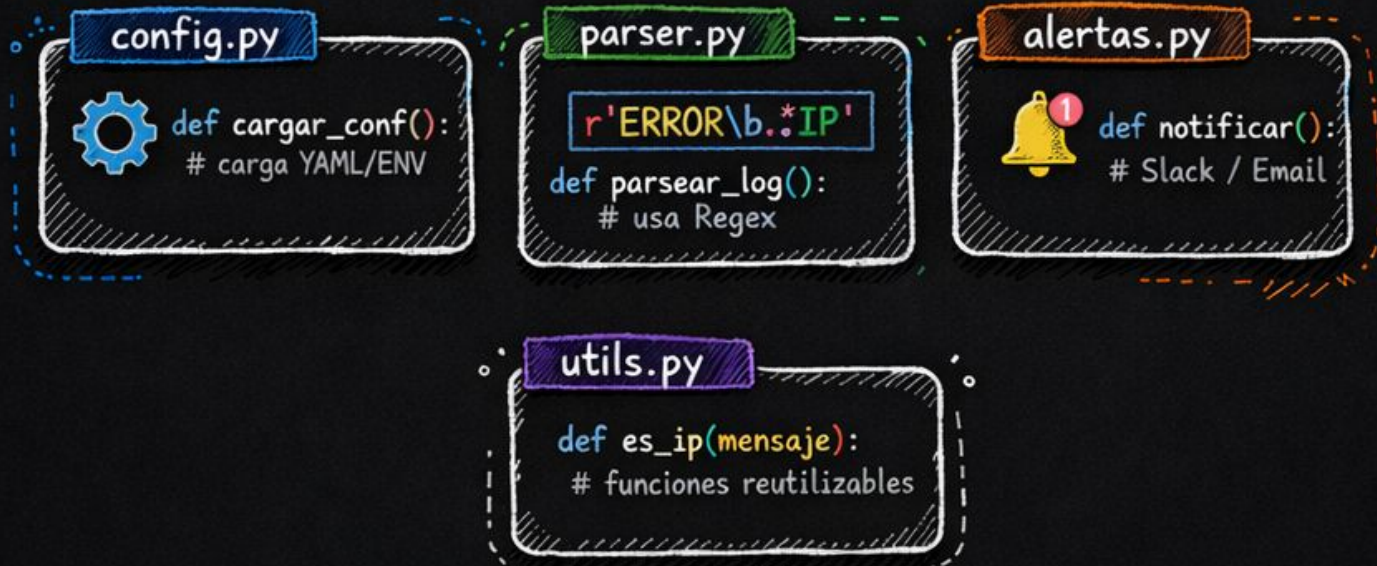


Script Monolítico

MODULARIDAD



MODULARIDAD





services > soar >  app.py

```
from soc import approval, notify, reporting, store
from soc.agent import investigate
from soc.enrichment import enrich_ip
from soc.models import Alert, Incident
from soc.pre_enrichment import apply_pre_enrichment, parse_enriched_payload
from soc.wazuh import alert_from_wazuh
```

soc >  store.py

```
...
def save(incident: Incident) -> None: ...
def load(incident_id: str) -> Incident | None: ...
def list_all() -> list[Incident]: ...
```




LOGGING



```
logging.basicConfig(level=logging.INFO, format="%(asctime)s [soar-bridge] %(message)s")  
log = logging.getLogger("soar-bridge")
```

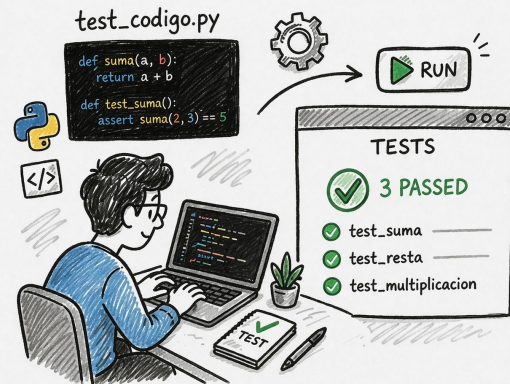
```
log.info("Playbook start %s (IP %s)", incident.id[:12], alert.src_ip)
```

TESTING

- Tests unitarios con pytest
- Mocking

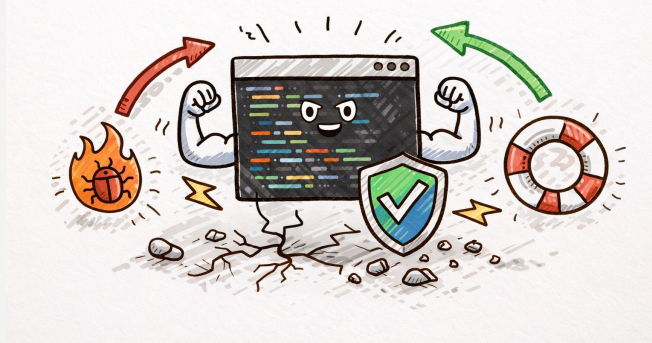
```
def test_wazuh_parser_extrae_ip_y_usuario():  
    payload = {  
        "id": "abc",  
        "rule": {"id": "5712", "description": "SSH brute force", "level": 10},  
        "data": {"srcip": "203.0.113.5", "dstuser": "root"},  
        "agent": {"name": "victim"},  
    }  
    alert = alert_from_wazuh(payload)  
    assert alert.src_ip == "203.0.113.5"  
    assert alert.user == "root"  
    assert alert.rule_id == "5712"
```

```
def test_mock_llm_pide_reputacion_primer():  
    tools = [{"function": {"name": n}} for n in ("check_ip_reputation", "get_user_login_history", "create_incident_ticket")]  
    resp = MockLLM().chat(messages=[], tools=tools, context={"alert": {"src_ip": "1.2.3.4", "user": "root"}})  
    assert resp.tool_calls[0].name == "check_ip_reputation"  
    assert resp.tool_calls[0].arguments["ip"] == "1.2.3.4"
```





RESILIENCIA



```
try:
    mail_result = notify.send(incident)
except Exception as exc: # noqa: BLE001
    mail_result = f"notificación falló: {exc}"
    log.exception("SMTP/notify falló para %s", incident.id[:12])
incident.log(mail_result)
```



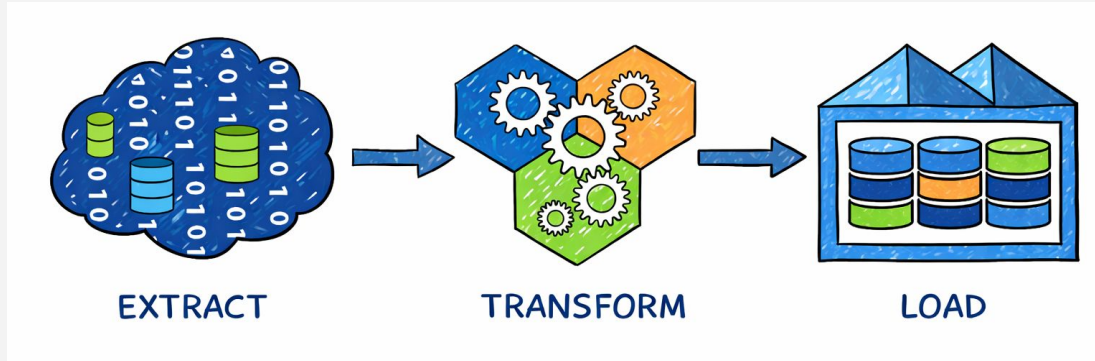
“

*Un script útil resuelve un problema.
Un buen código evita crear uno nuevo.*

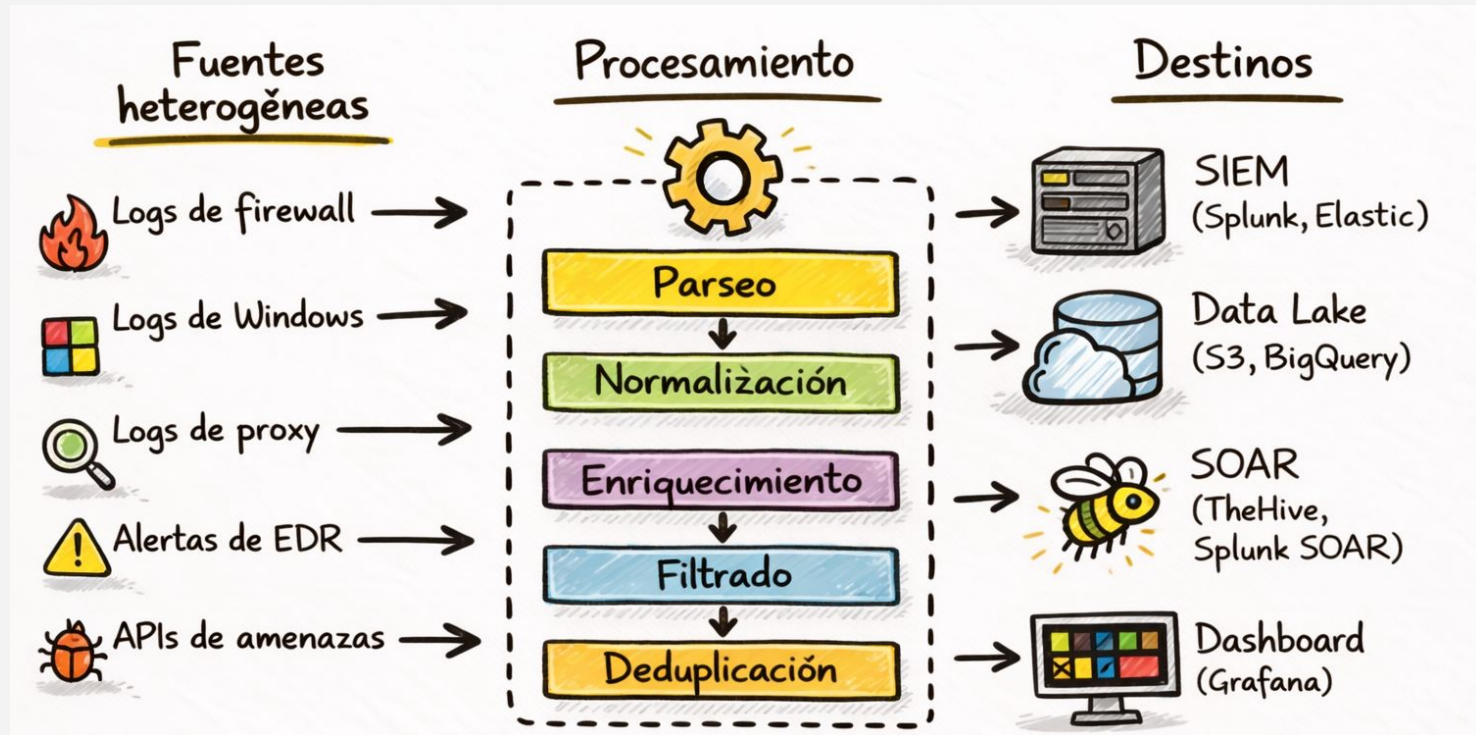
PIPELINES ETL E IOCS



¿QUÉ ES ETL?



¿QUÉ ES ETL?



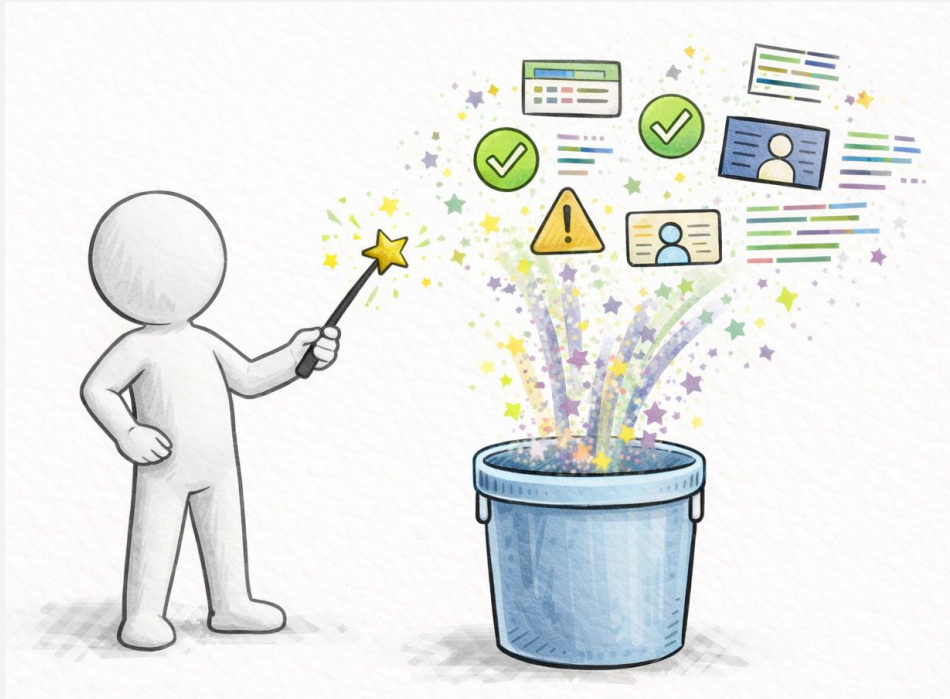
EXTRACT



Fuentes de datos del SOC

- archivos de logs locales
- APIs de SIEM/EDR
- Colas de mensajes (SQS / Kafka)
- Data Lakes (s3 / parquet)

TRANSFORM



Convertir ruido en datos
utilizables

- parseo
- normalización
- enriquecimiento
- deduplicación

TRANSFORM: PARSEO

Objetivo: Ordenar datos



TRANSFORM: PARSEO

Ejemplo: parsear evento windows

```
"@timestamp": "2024-03-15T08:42:17.503Z",
"winlog": {
  "event_id": 4625,
  "channel": "Security",
  "computer_name": "DESKTOP-ABC123",
  "provider_name": "Microsoft-Windows-Security-Auditing",
  "record_id": 8821043,
  "task": "Logon",
  "keywords": ["Audit Failure"],
  "user": {
    "name": "jsmith",
    "domain": "CORP",
    "type": "User"
  },
  "event_data": {
    "LogonType": "3",
    "FailureReason": "Unknown user name or bad password",
    "IpAddress": "192.168.10.45",
    "IpPort": "51322"
  }
},
"process": { "name": "lsass.exe", "pid": 748 },
"host": {
  "os": { "name": "Windows Server 2019" },
  "ip": ["10.0.1.15"]
},
"message": "An account failed to log on. Subject: SYSTEM. Logon Type: 3. Account: jsmith.",
"agent": {
  "version": "8.12.0",
  "type": "winlogbeat"
}
```

↓ parsear_windows_event(evento_json) ↓

SALIDA — DICCIONARIO NORMALIZADO

PARSEADO

```
"timestamp": "2024-03-15T08:42:17.503Z"
"event_id": 4625
"channel": "Security"
"computer": "DESKTOP-ABC123"
"usuario": "jsmith"
"proceso": "lsass.exe"
"mensaje": "An account failed to log on. Subject: SYSTEM. Logon Type: 3. Account: jsmith."
"raw": { ... objeto JSON completo original ... }
```

El campo "raw" conserva el JSON original completo — útil para enrichment posterior sin reparsear.

TRANSFORM: PARSEO

Ejemplo: de una línea de syslog sacar la fecha, el host, el proceso y el mensaje.

ENTRADA — LOG CRUDO SIN PARSEAR

```
Mar 15 08:42:17 web-server-01 sshd[1234]: Failed password for invalid user admin from 192.168.1.50 port 43210 ssh2
```

PARTES IDENTIFICADAS POR EL PATRÓN REGEX

mes: Mar dia: 15 hora: 08:42:17 host: web-server-01 proceso: sshd[1234]

mensaje: Failed password for invalid user admin from 192.168.1.50 port 43210 ssh2

↓ parsear_syslog(linea) ↓

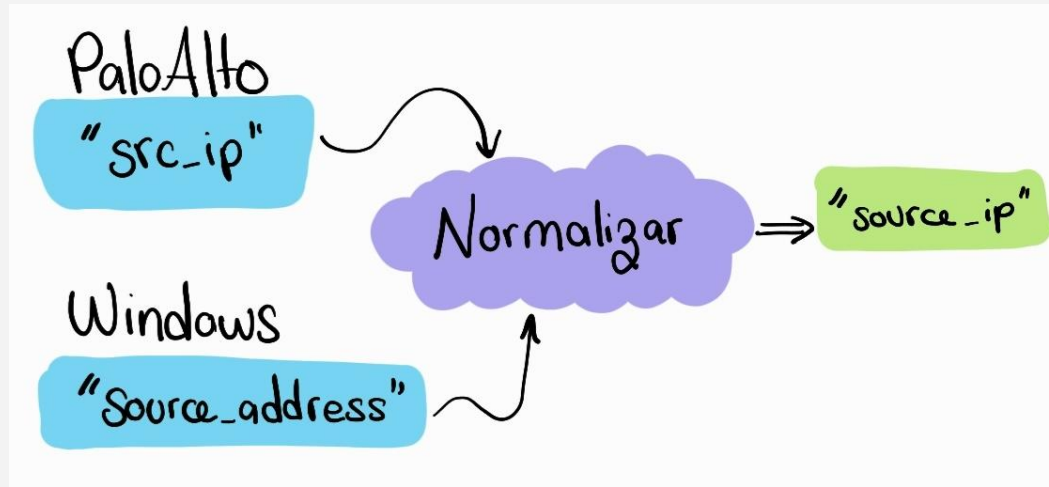
SALIDA — DICCIONARIO ESTRUCTURADO PARSEADO

```
{
  "timestamp": "2024-Mar-15 08:42:17",
  "hostname": "web-server-01",
  "proceso": "sshd[1234]",
  "mensaje": "Failed password for invalid user admin from 192.168.1.50 port 43210 ssh2"
}
```

TRANSFORM: NORMALIZACIÓN

Objetivo: Convertir a esquema común

Estándares de la industria: ECS y OCSF



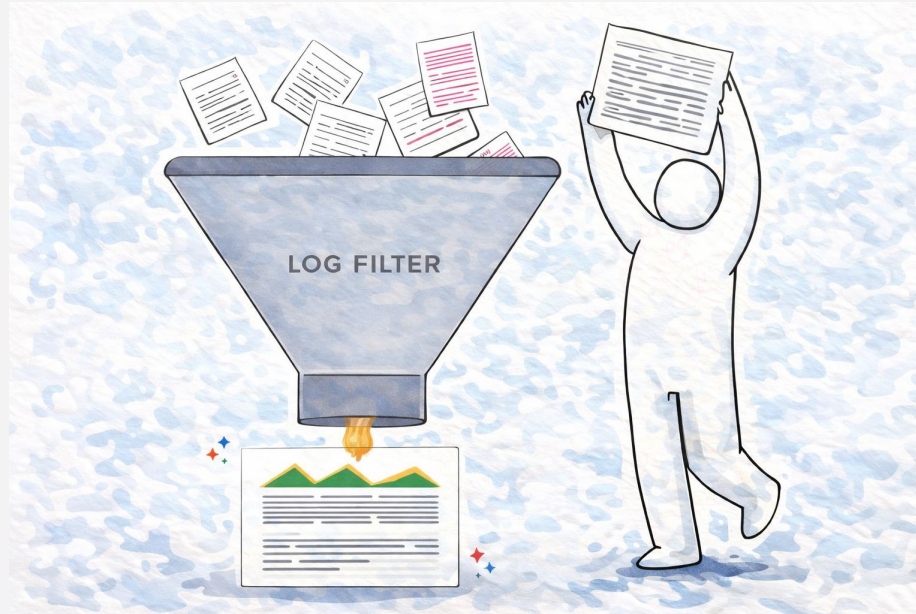
TRANSFORM: ENRIQUECIMIENTO

Objetivo: agregar contexto



TRANSFORM: FILTRADO Y DEDUPLICACIÓN

Objetivo: eliminar lo que no nos interesa y los duplicados



LOAD

Llevar los logs a donde deberían estar



ETL en un pipeline de respuesta

1 EXTRACT

Sacamos la alerta del SIEM
(Wazuh, JSON crudo)

2 TRANSFORM

Normalizar + Enriquecer

3 LOAD

Guardar en el destino:

- /data/state/incidents/*.json
- mail
- ticket MCP

EXTRACCIÓN DE IOCS

Regex - scripts - pipelines de enriquecimiento



¿QUÉ SON LOS IOCS?

Indicator of Compromise (IOC)

Dato técnico que indica que un sistema fue o podría estar comprometido. Es un artefacto observable y verificable.



ALGUNOS PATRONES REGEX PARA IOCS

IPv4

```
^r"\b(?:25[0-5]|2[0-4]\d|[01]?\d\d?)\.\.}{3}\. "$
```

Hash SHA1

```
^r"\b[0-9a-fA-F]{40}\b "$
```

URL

```
^r"https?://[a-zA-Z0-9\-.~:/?#\[\]@!$&'()*+,\;=_%]"
```

CVE

```
^r"\bCVE-\d{4}-\d{4,7}\b "$
```

Hash MD5

```
^r"\b[0-9a-fA-F]{32}\b "$
```

Hash SHA256

```
^r"\b[0-9a-fA-F]{64}\b "$
```

Email

```
^r"\b[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}\b "$
```

MITRE ATT&CK

```
^r"\bT\d{4}(?:\.\d{3})?\b "$
```



REGEX Expresiones regulares.

```
import re

# Regex para pescar una IPv4 del texto crudo si Wazuh no la trae estructurada.
_IP_RE = re.compile(r"\b(?:\d{1,3}\.){3}\d{1,3}\b")

def _first_ip(text: str) -> str | None:
    m = _IP_RE.search(text or "")
    return m.group(0) if m else None
```

```
# La IP de origen puede venir en distintos campos según la regla.
src_ip = (
    data.get("srcip")
    or data.get("src_ip")
    or _first_ip(payload.get("full_log", ""))
    or "0.0.0.0"
)
```

EXTRAER IOCS + ENRIQUECER



```
def enrich_ip(ip: str) -> dict:
    return {
        "ip_reputation": lookup_ip_reputation(ip),
        "geolocation": geolocate_ip_offline(ip),
        "source": "soc-worker-enrichment",
        "note": "En producción esto sería VirusTotal / AbuseIPDB.",
    }
```

librerías como **ioc-finder**

librerías externas		regex	
	Conveniente		Control total
	Rápido de implementar		Personalizable
	Bien testado		Sin dependencias externas
	Menos control sobre la lógica		Más trabajo inicial

<https://pypi.org/project/ioc-finder>

```
pip install ioc-finder
```

FUNDAMENTOS DE SOAR

RUNBOOK VS PLAYBOOK

RUNBOOK

Texto que describe pasos — lo consulta el analista

PASO 1: Obtener información del incidente

- Registra la IP origen
- Registra el usuario atacado
- Registra los intentos fallidos

PASO 2: Verificar si la IP es conocida

- Busca en CMDB e historial

PASO 3: Tomar acciones de mitigación

- Bloquea la IP en el firewall
- Fuerza cambio de contraseña

PASO 4: Documentar y escalar

- Crea ticket y notifica equipo



PLAYBOOK

Código ejecutable — corre solo en 2-3 segundos

```
def responder_bruta_rdp(ip, usuario, intentos):
    incidente = crear_caso("RDP BruteForce",
                          ip=ip, usuario=usuario)

    # ¿IP conocida? → falso positivo probable
    if consultar_cmdb(ip):
        return "Falso positivo - IP conocida"

    # Ejecutar acciones automáticamente
    bloquear_ip_en_firewall(ip)
    forzar_cambio_password(usuario)

    # Escalar si hay acceso exitoso
    if historial_tiene_acceso(usuario):
        incidente.prioridad = "CRITICAL"

    crear_ticket_jira(incidente)
    return incidente # En 2-3 segundos ✓
```

SOAR

Security Orchestration,
Automation and Response

CICLO DE VIDA DE INCIDENTE

1 DETECCIÓN

Alerta llega desde SIEM, EDR o firewall. SOAR la recibe automáticamente.

2 TRIAJE AUTO

Enriquece información. Calcula scoring de riesgo.

3 DECISIÓN

Alto → caso HIGH.
Medio → pausa para humano.
Bajo → resuelve automáticamente.

4 RESPUESTA

La respuesta programada para caso:
Phishing → bloquear.
Malware → aislar.
Fuerza bruta → bloquear IP.

5 REVISIÓN

Si no se cerró el caso:
analista revisa, aprueba o ajusta.

6 CIERRE

Documentar, análisis post-incidente, actualizar playbooks.

Tipos de plataformas SOAR



Comerciales
Enterprise



Cloud



Open source,
autohosteadas

INTEGRACIÓN CON EL STACK

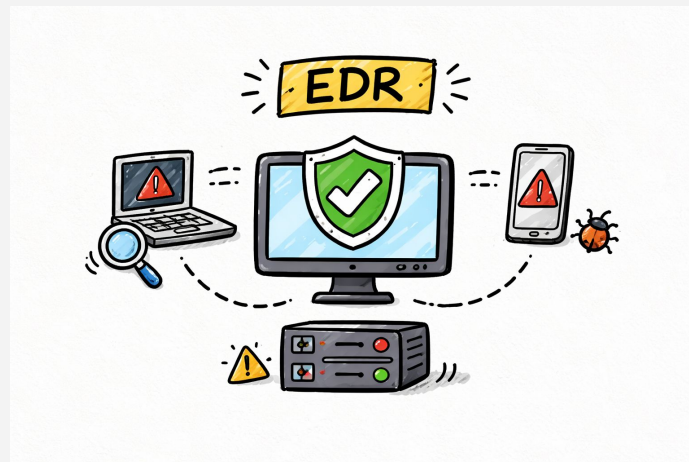
El objetivo es integrar las herramientas de seguridad y orquestar la respuesta entre ellas



INTEGRACIÓN CON EDR

Queremos:

- Recibir alertas del endpoint.
- Consultar historial del dispositivo.
- Aislar endpoints comprometidos.
- Ejecutar acciones remotas

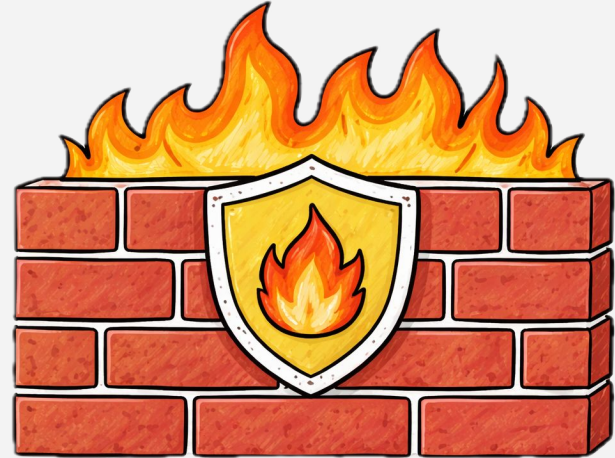




INTEGRACIÓN CON FIREWALLS Y CONTROLES DE RED

Queremos:

- Bloquear IPs o dominios sospechosos
- Crear reglas temporales
- Ajustar políticas de acceso
- Redirigir tráfico a análisis

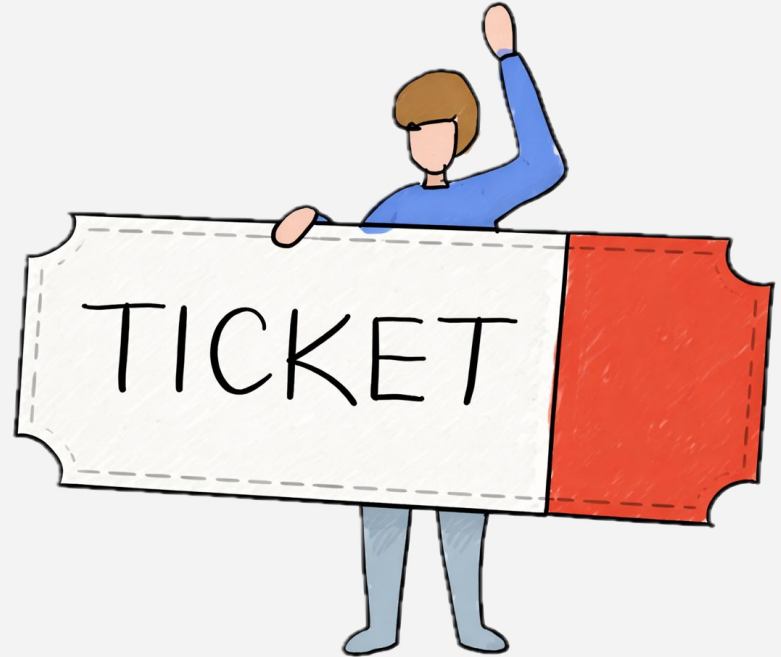




INTEGRACIÓN CON TICKETING

Queremos:

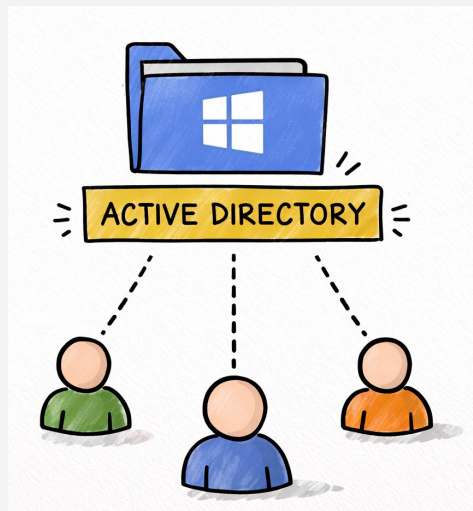
- Crear tickets automáticamente
- Actualizar estados de incidentes
- Adjuntar evidencias
- Escalar o cerrar casos



INTEGRACIÓN CON ACTIVE DIRECTORY

Queremos:

- Consultar datos del usuario
- Resetear contraseñas
- Deshabilitar cuentas
- Revocar accesos



SCORING DE RIESGO



SCORING DE RIESGO

- Muchas alertas por día
- Tenemos que priorizar

Queremos:

- Asignar un número que indique qué tan probable es que una alerta sea una amenaza real.

3 DECISIÓN

Alto → caso HIGH.
Medio → pausa para humano.
Bajo → resuelve automáticamente.

¿Qué tener en cuenta para el scoring?

Alertas

- Tipo de eventos
- Severidad
- IOCs presentes

Usuarios

- Rol
- Historial de alertas
- Comportamiento típico

El activo

Servidor de producción vs. equipos de pruebas

Contexto

- Hora
- Ubicación geográfica
- Datos accedidos

Threat Intel

- IP en blocklist
- Patrón coincide con ataque conocido

Scoring de riesgo

Alertas

- Tipo de eventos
- Severidad
- IOC's presentes

Usuarios

- Rol
- Historial de alertas
- Comportamiento típico

El activo

Servidor de producción vs.
equipos de pruebas

Contexto

- Hora
- Ubicación geográfica
- Datos accedidos

Threat Intel

- IP en blocklist
- Patrón coincide con ataque conocido

scoring.py

```
def compute_risk_score(incident: Incident) -> dict[str, Any]:
    """
    Calcula score 0-100 y breakdown por categoría.
    El playbook usa el total para priorizar y decidir acción.
    """
    categories = {
        "alerta": _score_alert(incident),
        "usuario": _score_user(incident),
        "activo": _score_asset(incident),
        "contexto": _score_context(incident),
        "threat_intel": _score_threat_intel(incident),
    }

    breakdown: dict[str, Any] = {}
    total = 0
    all_reasons: list[str] = []
    for name, (pts, reasons) in categories.items():
        breakdown[name] = {"points": pts, "reasons": reasons}
        total += pts
        all_reasons.extend(reasons)

    total = min(100, total)

    if total >= THRESHOLD_HIGH:
        risk = "high"
    elif total >= THRESHOLD_MEDIUM:
        risk = "medium"
    else:
        risk = "low"

    return {
        "score": total,
        "risk": risk,
        "breakdown": breakdown,
        "reasons": all_reasons,
        "thresholds": {"high": THRESHOLD_HIGH, "medium": THRESHOLD_MEDIUM},
    }
```

“

Los runbooks nos dan estructura y los playbooks nos permiten ejecutar esa estructura.

Y el SOAR nos permite conectar herramientas y automatizar tareas.

INTEGRACIÓN DE CONECTORES

¿CON QUÉ HERRAMIENTAS SE CONECTA UN SOAR?

Categoría	Herramientas típicas	¿Qué hace el SOAR?
EDR	CrowdStrike, Defender	Aislar endpoints, ejecutar comandos
Firewall	Palo Alto, Fortinet	Bloquear IPs y dominios
Ticketing	Jira, ServiceNow	Crear y actualizar incidentes
Threat Intel	VirusTotal, MISP	Enriquecer IOCs con contexto
Email	Exchange, Proofpoint	Poner en cuarentena phishing
Identidades	Active Directory	Deshabilitar usuarios comprometidos

EL PROBLEMA DEL CONECTOR AD-HOC

- Cada conector resuelve lo mismo de forma distinta
- Se repite autenticación, errores, timeouts y logging
- Los conectores se vuelven inconsistentes
- Mantenerlos escala mal



```
# virustotal.py    - 80 líneas, su propia auth
# crowdstrike.py  - 120 líneas, su propio retry
# jira.py         - 90 líneas, su propio timeout
# splunk.py       - 110 líneas, su propio logging
# palo_alto.py    - 100 líneas, sus propios errores
# ...15 conectores x patrones incompatibles
# Un bug de timeout → arreglarlo en 15 archivos ☠
```

LA SOLUCIÓN: UNA INTERFAZ COMÚN

- Centralizamos las operaciones comunes
 - Autenticación
 - Manejo de errores
 - Logging
 - Formato estándar de respuesta
- Lo específico queda dentro de cada integración
- Agregar nuevos conectores es más simple

```
class ConectorBase:
    def ejecutar(self, accion):
        print("logging / validación / errores")
        return self._ejecutar(accion)

    def _ejecutar(self, accion):
        raise NotImplementedError

class ConectorEDR(ConectorBase):
    def _ejecutar(self, accion):
        return f"EDR: {accion}"
```



LA SOLUCIÓN: UNA INTERFAZ COMÚN

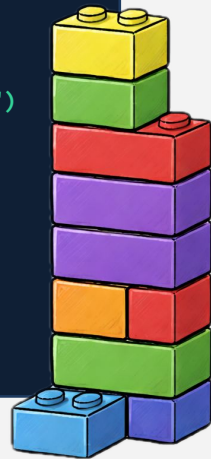
Cada conector implementa:

- Cómo hablar con su API
- Qué acciones soporta
- Cómo interpreta las respuestas
- Cómo traduce resultados al formato común

```
class ConectorBase:
    def ejecutar(self, accion):
        print("logging / validación / errores")
        return self._ejecutar(accion)

    def _ejecutar(self, accion):
        raise NotImplementedError

class ConectorEDR(ConectorBase):
    def _ejecutar(self, accion):
        return f"EDR: {accion}"
```

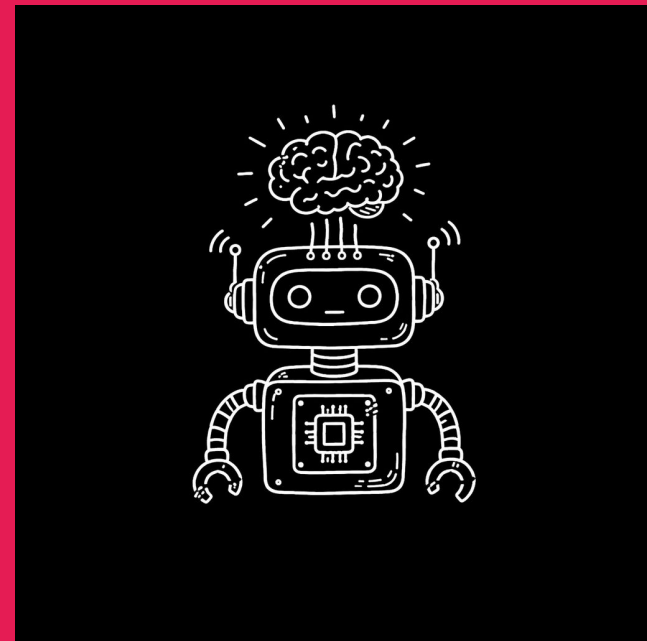


Cómo aplicamos el mismo concepto en el lab...

```
class LLMBackend(Protocol):  
    # Contrato que cumplen los tres backends.  
    def chat(  
        self, messages: list[dict], tools: list[dict], context: Optional[dict] = None  
    ) -> LLMResponse: ...
```

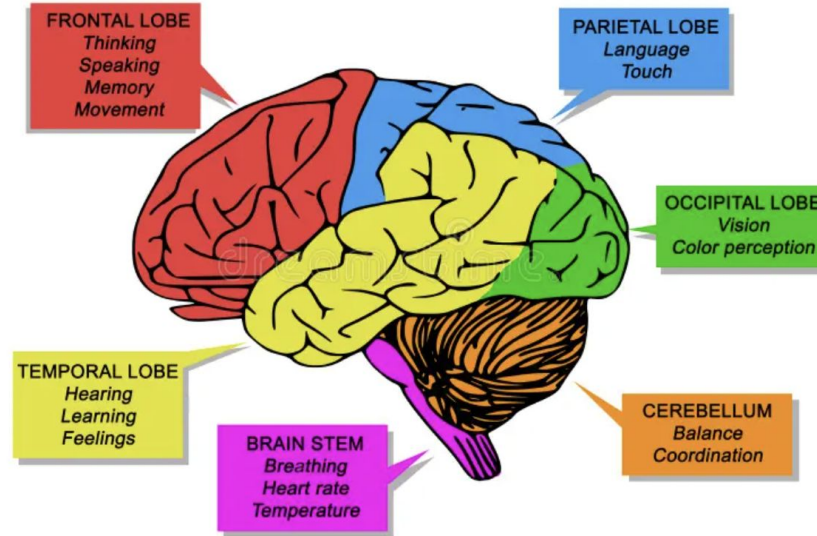
```
def get_backend() -> LLMBackend:  
    # Fábrica: elige el backend según la config. Default seguro = mock.  
    backend = settings.llm_backend.lower()  
    if backend == "ollama":  
        return OllamaLLM()  
    if backend == "openai":  
        return OpenAILLM()  
    return MockLLM()
```

INTRODUCCIÓN A IA Y A MACHINE LEARNING



QUÉ ES LA IA?

El campo que engloba cualquier programa que intente replicar aspectos de la inteligencia humana...

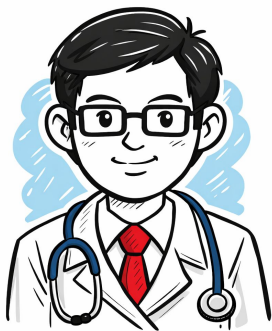


QUÉ ES LA IA?

El campo que engloba cualquier programa que intente replicar aspectos de la inteligencia humana...

Incluyendo programas basados en reglas (sistemas expertos).

Ej: sistema de diagnóstico de enfermedades



Reglas del sistema	
SI fiebre + tos + dolor de cuerpo + fatiga	→ Gripe (Influenza)
SI congestión + estornudos + dolor de garganta	→ Resfriado común
SI fiebre + dolor de garganta	→ Posible faringitis
SI tos + fatiga	→ Posible bronquitis
SI congestión + estornudos (sin fiebre)	→ Posible alergia



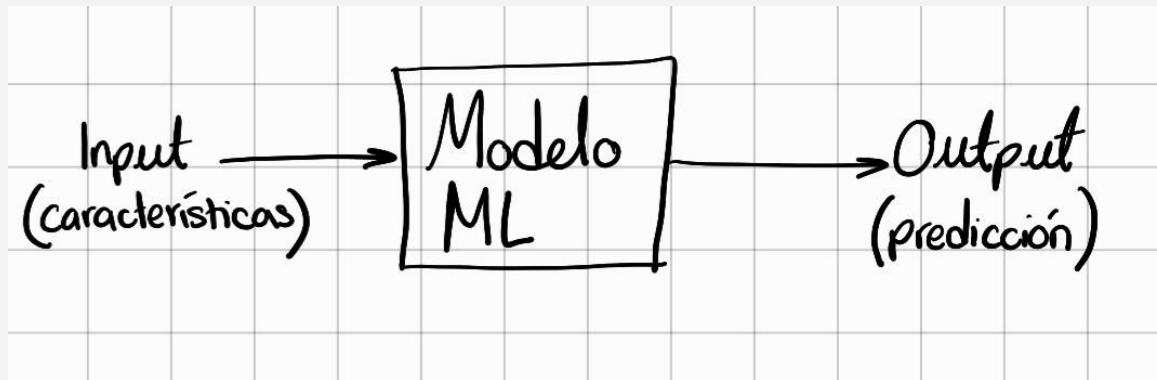
PROGRAMAS ESCRITOS CON REGLAS

Antes del machine learning, todos los programas de detección tenían que ser programados explícitamente con reglas, línea por línea: "si esto entonces aquello"



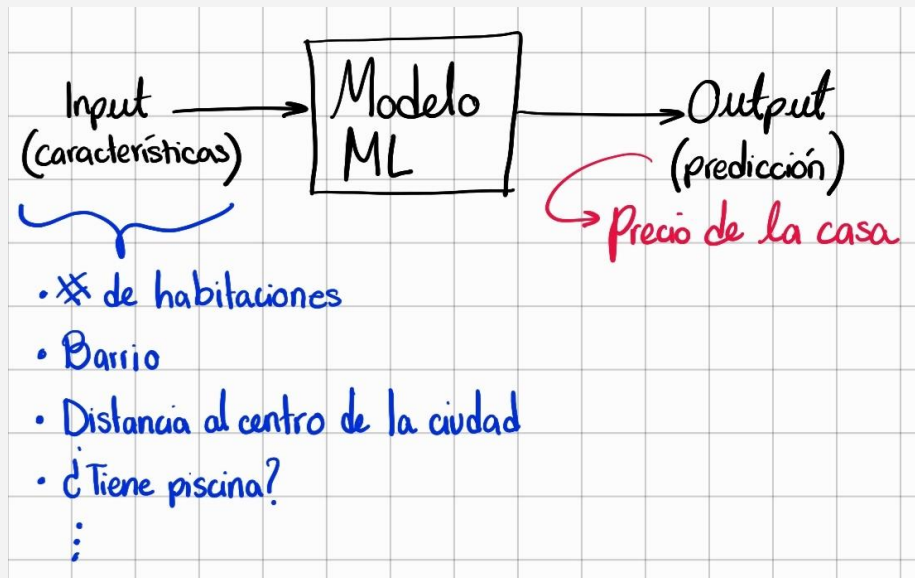
Modelo de Machine Learning

El objetivo es construir un modelo que pueda predecir algo (output) basándose en datos de entrada (input)



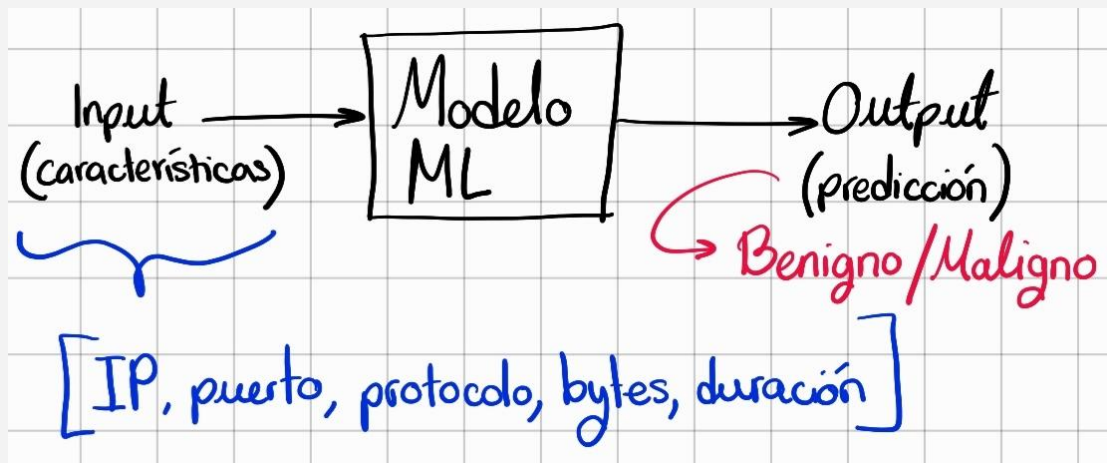
Modelo de Machine Learning

Ejemplo: modelo para predecir el precio de una casa.

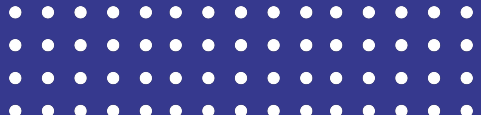


Modelo de Machine Learning

Ejemplo: modelo para predecir si un evento en el tráfico de red es tráfico normal o un posible ataque.



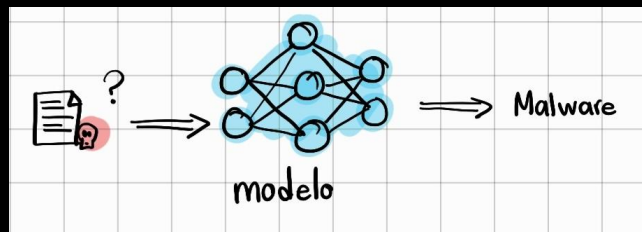
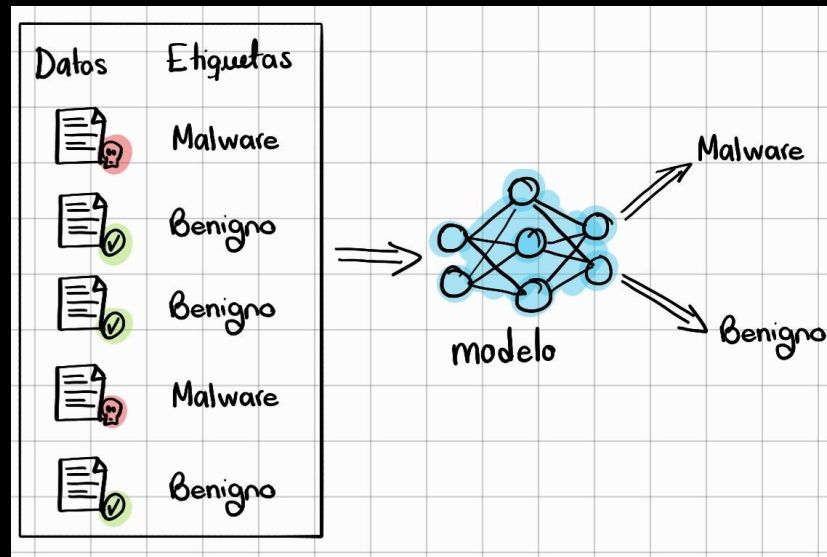
APRENDIZAJE SUPERVISADO VS. NO SUPERVISADO



Aprendizaje Supervisado

El dataset de entrenamiento contiene ejemplos **etiquetados** (con respuestas correctas ya conocidas) de lo que queremos detectar.

El modelo aprende cuáles características (**features**) tienen más peso para hacer una predicción correcta.



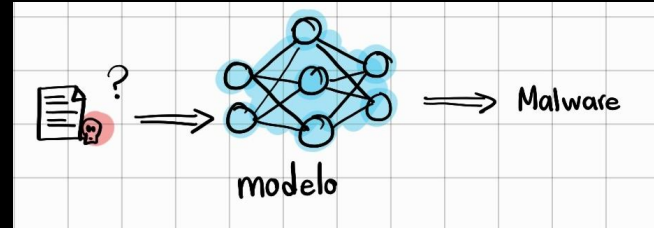
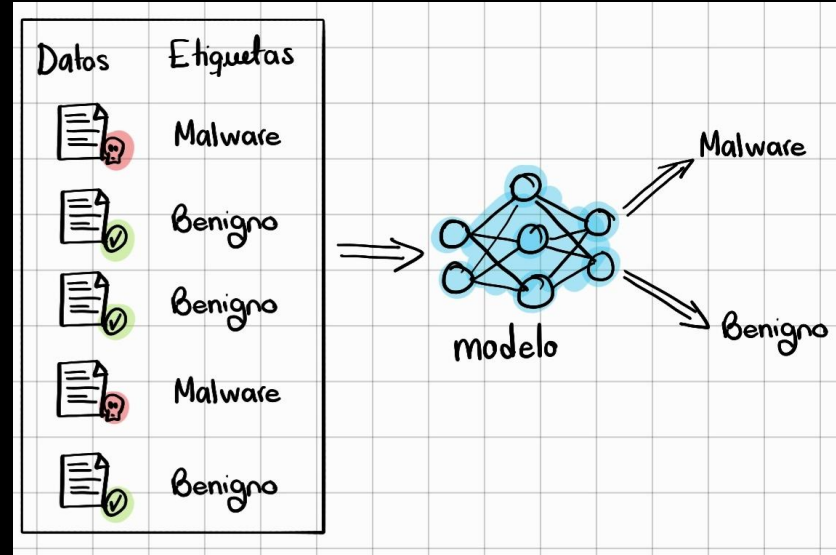
Aprendizaje Supervisado

Ventajas

- Efectivo si las etiquetas son correctas

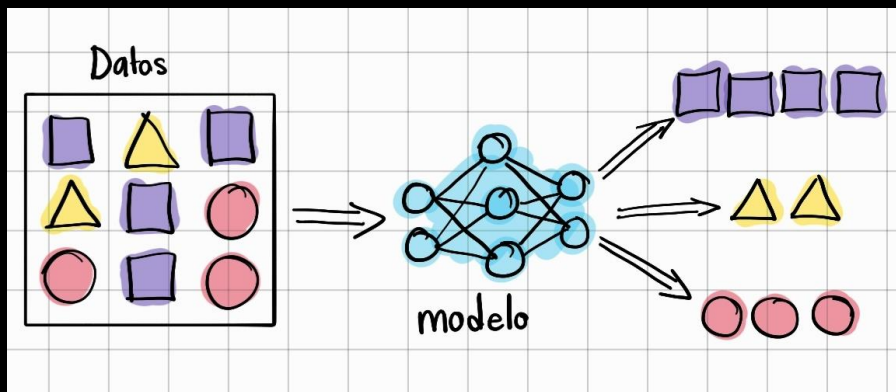
Desventajas

- Requiere etiquetado manual (costoso)
- No detecta variantes nunca antes vistas



Aprendizaje No Supervisado

- El dataset de entrenamiento no tiene etiquetas
- El modelo intenta encontrar patrones ocultos en los datos: busca similitudes y las agrupa.
- Cuando un dato nuevo llega, indica a qué grupo pertenece.



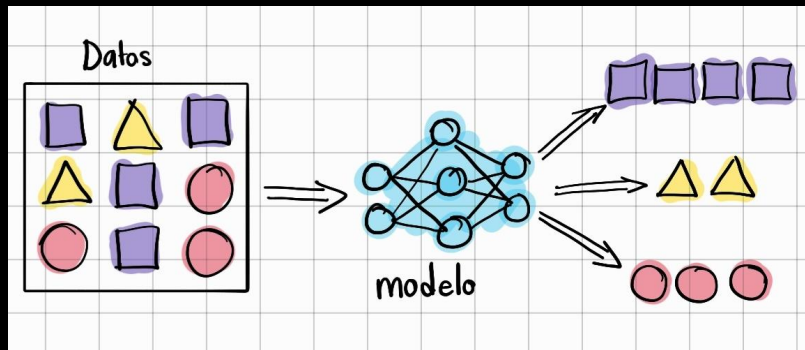
Aprendizaje No Supervisado

Ventajas:

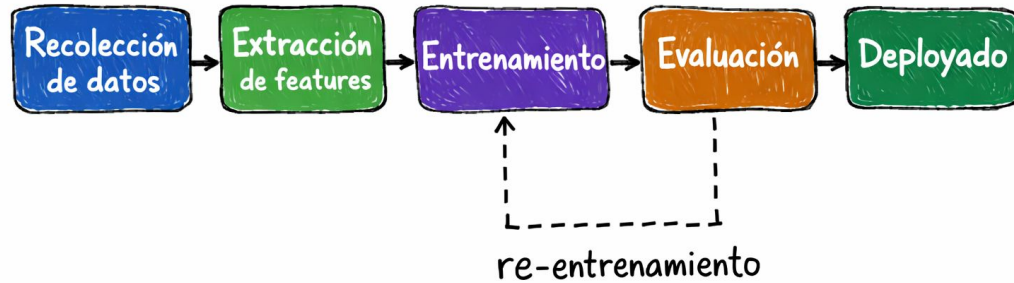
- No requiere etiquetado manual
- Puede detectar anomalías o patrones inesperados

Desventajas:

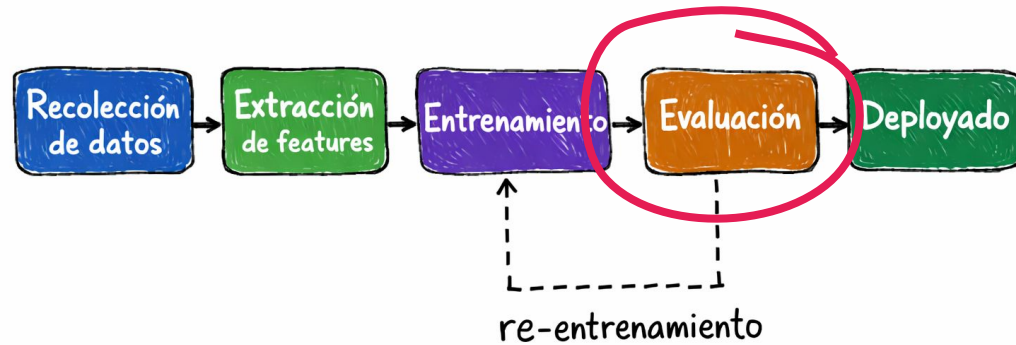
- Es difícil validar si el resultado es correcto
- Los clusters pueden no corresponder a “malware vs benigno”



Workflow para armar un modelo de Machine Learning



Workflow para armar un modelo de Machine Learning



Métricas de evaluación

Accuracy

Predicciones correctas / total.

Precision

De los que dije 'malware', ¿cuántos lo son realmente? Reduce falsos positivos.

Recall

De todo el malware real, ¿cuánto detecté? Reduce falsos negativos.

- TP: Verdaderos Positivos
- TN: Verdaderos Negativos
- FP: Falsos Positivos
- FN: Falsos Negativos

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

Ajuste continuo del modelo

- Periódicamente (mensualmente o trimestralmente), **reunís nuevos datos** que fueron etiquetados por analistas y **reentrenás** el modelo.
- Esto asegura que el modelo se adapte a nuevas amenazas y características de tu entorno específico.

Caso #123:

Predicción: Score 85 (CRITICAL)

Veredicto analista: Falso positivo (fue error de configuración)

→ Modelo fue incorrecto, agregar caso en el dataset para re-entrenar

Caso #456:

Predicción: Score 25 (LOW)

Veredicto analista: Verdadero ataque (después de investigación)

→ Modelo falló, agregar caso en el dataset para re-entrenar

Limitaciones del ML en seguridad

Cold Start Problem

No se puede entrenar para detectar amenazas nuevas sin ejemplos previos.

Ataques Adversariales

El atacante puede manipular su malware para evadir el modelo al conocer qué características detecta.

Garbage In, Garbage Out

Datos de entrenamiento incorrectos o sesgados producen un modelo incorrecto o sesgado.

Falta de Explicabilidad

Las redes neuronales son 'cajas negras': dicen 'malware' sin explicar por qué. Los analistas necesitan el razonamiento.

Concept Drift

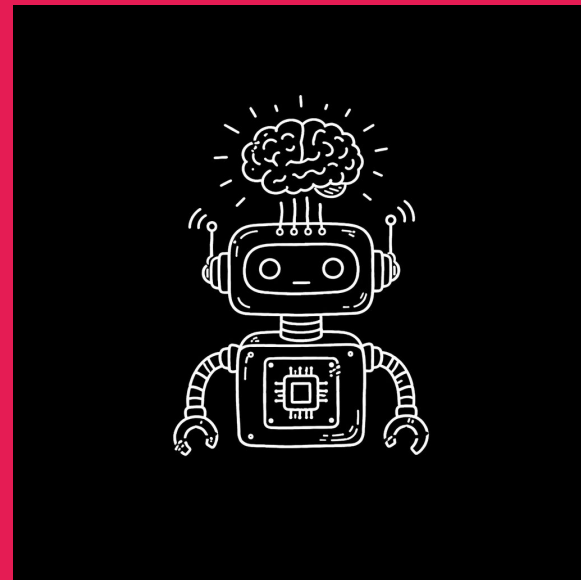
Si el malware evoluciona (y siempre evoluciona), el modelo entrenado en 2020 falla en 2024.

Alto Costo de Falsos Positivos

1% de FP sobre millones de archivos = cientos de alertas falsas diarias → analistas abrumados.

Solución: combinar ML + reglas de negocio + revisión humana (human-in-the-loop) + monitoreo de concept drift

ML EN SEGURIDAD Y DETECCIÓN DE ANOMALÍAS



Algoritmos comunes en seguridad

Árboles

- Árbol de Decisión – reglas binarias, fácil de interpretar
- Random Forest – múltiples árboles, reduce sobreajuste

Modelos Lineales

- Regresión Logística – clasificación binaria simple
- SVM – frontera óptima que maximiza separación

Redes Neuronales

- DNN – múltiples capas, muy potentes
- Requieren grandes datos y tiempo de entrenamiento

Anomalías

- Isolation Forest – aísla comportamientos raros
- Autoencoders – aprenden a reconstruir lo normal
- One-Class SVM – entrena solo con datos normales

Clustering

- K-Means – grupos por similitud
- DBSCAN – densidad variable, outliers = sospechoso

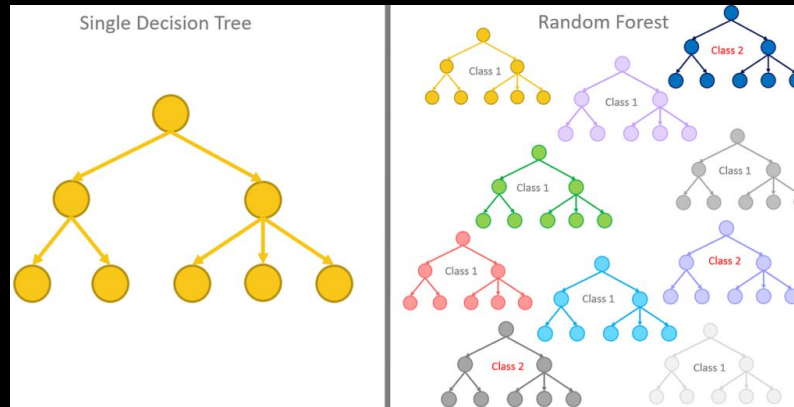
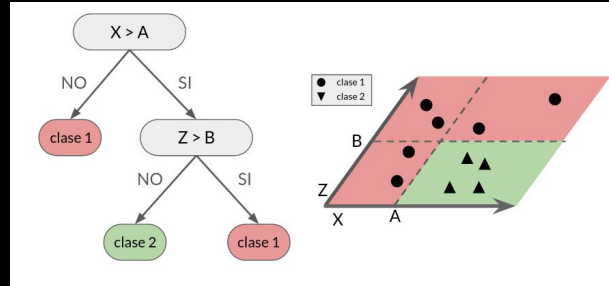
Dimensionalidad

- PCA – reduce variables, mantiene información
- t-SNE / UMAP – visualizar clústers en 2D

Algoritmos comunes en seguridad

Árboles

- Árbol de Decisión – reglas binarias, fácil de interpretar
- Random Forest – múltiples árboles, reduce sobreajuste



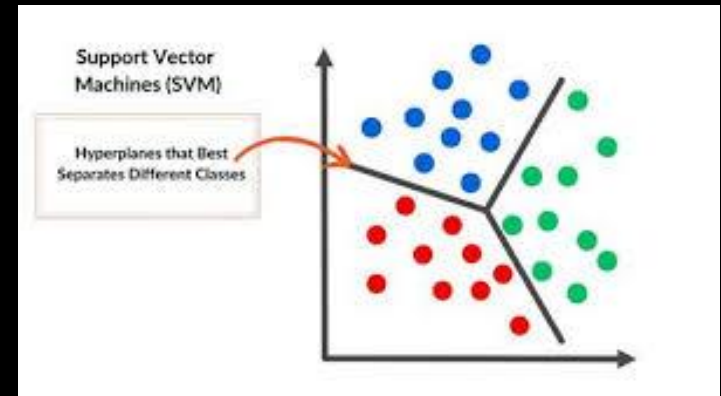
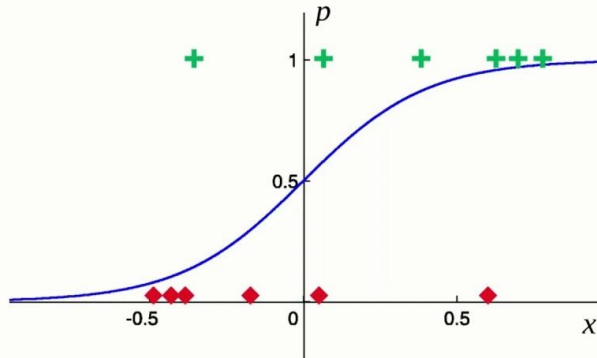
Algoritmos comunes en seguridad

Modelos Lineales

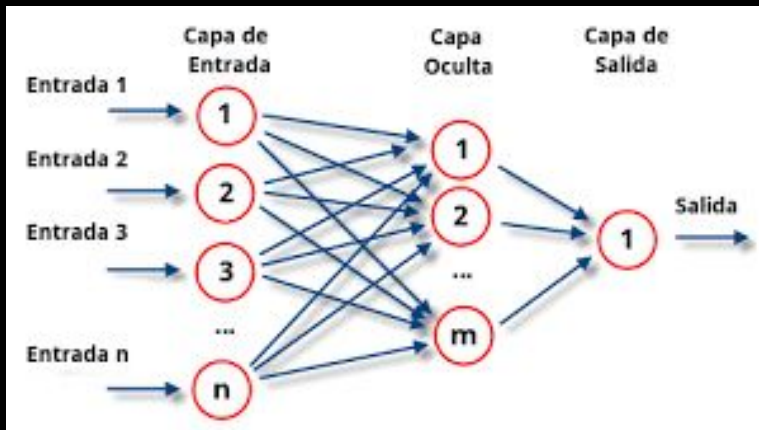
- Regresión Logística – clasificación binaria simple
- SVM – frontera óptima que maximiza separación

$$\log \frac{p}{1-p} = \beta_0 + \beta_1 x$$

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$



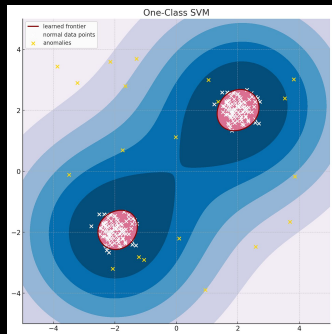
Algoritmos comunes en seguridad



Redes Neuronales

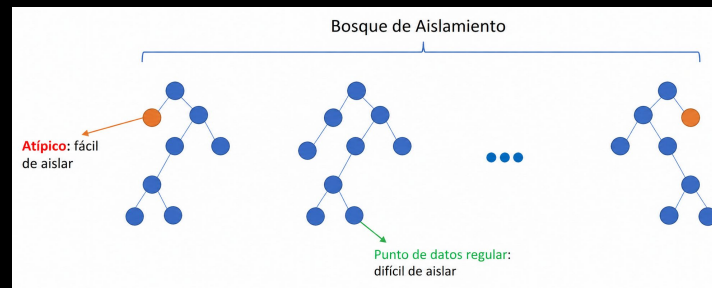
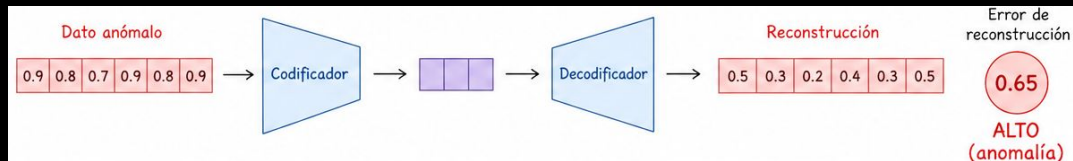
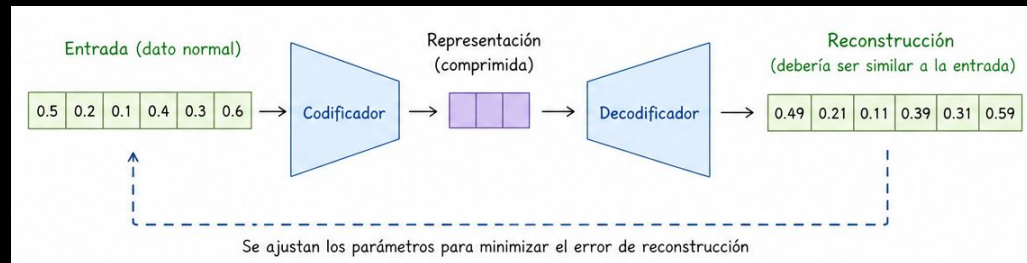
- DNN – múltiples capas, muy potentes
- Requieren grandes datos y tiempo de entrenamiento

Algoritmos comunes en seguridad



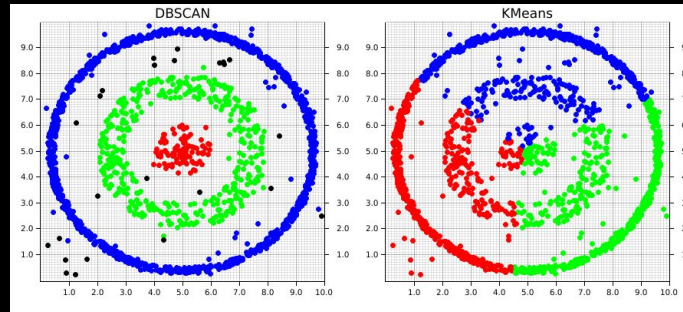
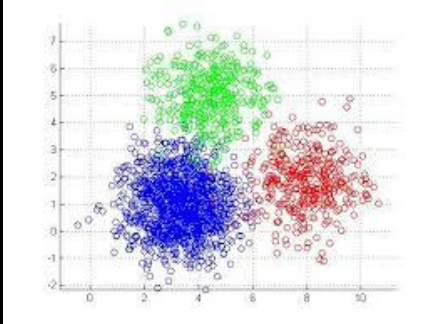
Anomalías

- Isolation Forest – aísla comportamientos raros
- Autoencoders – aprenden a reconstruir lo normal
- One-Class SVM – entrena solo con datos normales



Algoritmos comunes en seguridad

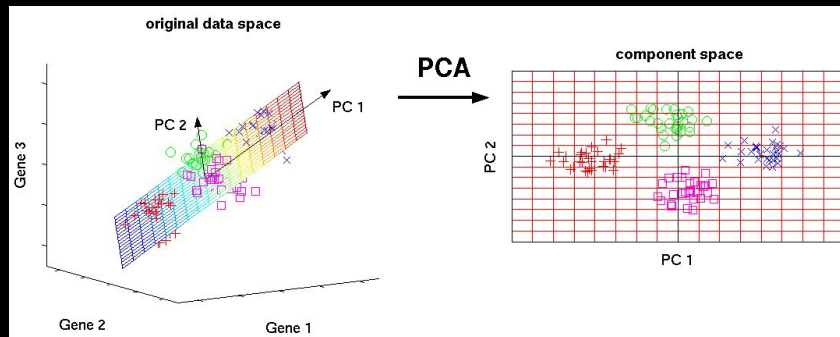
K-means



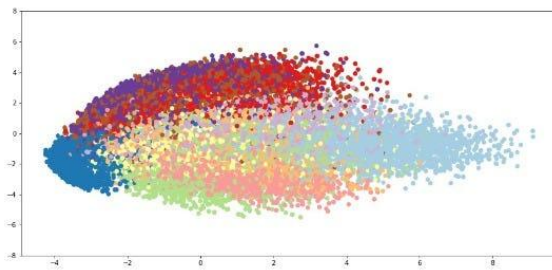
Clustering

- K-Means – grupos por similitud
- DBSCAN – densidad variable, outliers = sospechoso

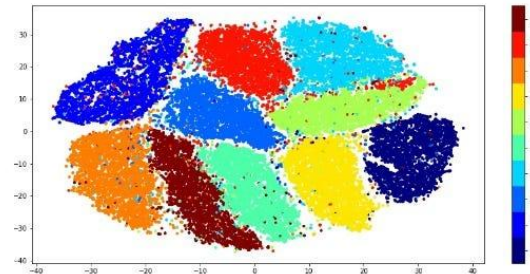
Algoritmos comunes en seguridad



MNIST - PCA



MNIST - TSNE



Dimensionalidad

- PCA – reduce variables, mantiene información
- t-SNE / UMAP – visualizar clústers en 2D

Algoritmos comunes en seguridad

Árboles

- Árbol de Decisión – reglas binarias, fácil de interpretar
- Random Forest – múltiples árboles, reduce sobreajuste

Modelos Lineales

- Regresión Logística – clasificación binaria simple
- SVM – frontera óptima que maximiza separación

Redes Neuronales

- DNN – múltiples capas, muy potentes
- Requieren grandes datos y tiempo de entrenamiento

Anomalías

- Isolation Forest – aísla comportamientos raros
- Autoencoders – aprenden a reconstruir lo normal
- One-Class SVM – entrena solo con datos normales

Clustering

- K-Means – grupos por similitud
- DBSCAN – densidad variable, outliers = sospechoso

Dimensionalidad

- PCA – reduce variables, mantiene información
- t-SNE / UMAP – visualizar clústers en 2D

Detección de anomalías



ANOMALY DETECTION

Algo que se desvía de lo normal.

Las reglas no escalan para esto porque “lo normal” es diferente para cada usuario, cada sistema, cada horario...

Análisis de Comportamiento de Usuarios (UBA)

UBA (User Behavior Analytics) analiza el comportamiento de los usuarios para detectar actividades sospechosas o inusuales.

¿Qué patrones analiza UBA?

Volumen

Cuántos datos mueve el usuario.
Normal: 50 registros/día
Sospechoso: 50,000 registros/día
de repente

Frecuencia

Con qué frecuencia realiza ciertas acciones.
Normal: 2-3 logins/día
Sospechoso: 50 logins en un día

Ubicación

Desde dónde se conecta el usuario.
Normal: oficina en Chicago
Sospechoso: acceso desde Beijing
sin previo aviso

Peer Groups

Comparación con usuarios del mismo rol.
Si todos en el grupo hacen X pero este usuario hace Y → alerta.
Perfil fijo o dinámico.

Secuencias Anómalas

No solo QUÉ hace el usuario sino en qué ORDEN.
Ej: crear cuenta → usarla → borrarla → repetir. Ningún motivo legítimo.

UEBA (User and Entity Behavior Analytics)

Es la evolución de UBA.

Agrega la letra “E” de Entidades (Todo lo que no es una persona pero genera actividad en la red):

- Servidores
- Routers y switched
- BBDD
- Aplicaciones
- Dispositivos IoT

UEBA + SIEM

- Reducir ruido
- Priorizar investigaciones
- Escalar con el tamaño

Fuentes de datos → SIEM → UEBA → Lista de usuarios/entidades más riesgosos → Investigación

Importancia de UEBA en el SOC

Sin UEBA, un analista tendría que revisar manualmente la actividad de miles de usuarios cada día. Eso es imposible.

Con UEBA, el analista recibe algo así:

"Top 10 usuarios más riesgosos hoy:"

1. Juan – riesgo: crítico
2. ...

Ahora el analista sabe exactamente dónde enfocar su tiempo e investigación.

“

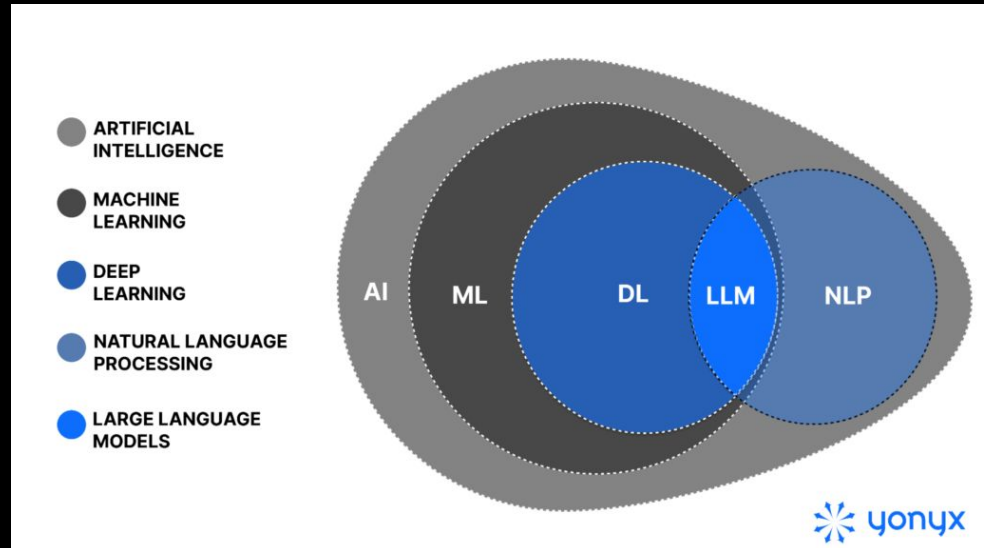
El ML permite detectar lo que las reglas no pueden: patrones ocultos, comportamientos anómalos y amenazas que nunca antes vimos.

NLP Y LLMS EN SEGURIDAD

¿QUÉ ES NLP?

Campo de la IA que permite a las máquinas entender y procesar lenguaje humano.

NLP $\neq$ LLMs



Por qué nos interesa aprender NLP para seguridad?

- En seguridad manejamos constantemente:
 - logs
 - alertas
 - tickets
 - reportes de malware
- NLP es el conjunto de técnicas para procesar ese texto:
 - clasificarlo
 - extraer información
 - detectar patrones

Clasificación con NLP

Aplicar una etiqueta a un texto.

Útil cuando las categorías se pueden inferir del contenido textual en sí (las palabras nos indican qué categoría va a ser)

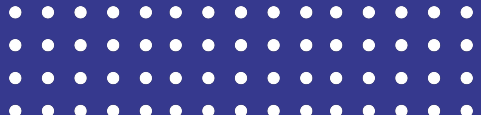
Ejemplos:

- Categorizar el mensaje de una alerta como fuerza bruta, exfiltración, reconocimiento
- Categorizar reportes de threat intel (ransomware, phishing, vulnerability, APT)
- Clasificar mails en spam/phishing/legítimo

Tokenización



¿Cómo funciona un LLM?





En 2017 un equipo de Google publicó un paper: “Attention is all you need”

Qué introdujo la arquitectura de IA “Transformer”

Arquitectura Transformer

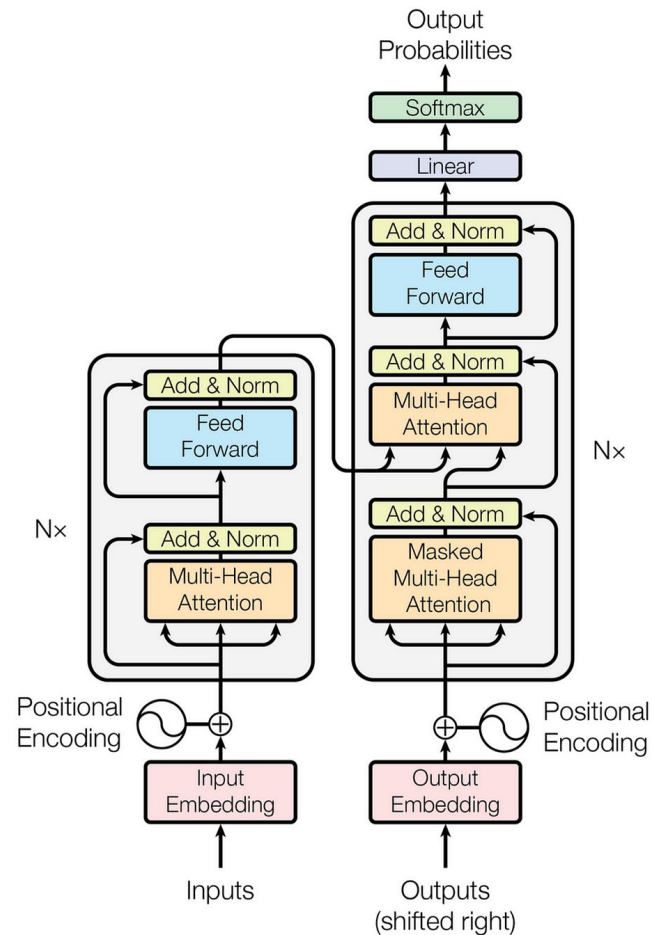
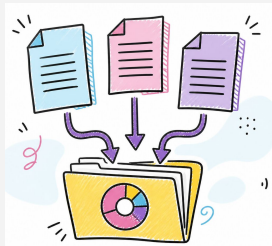


Figure 1: The Transformer - model architecture.

¿Qué necesitamos para hacer un LLM?

Muchos datos

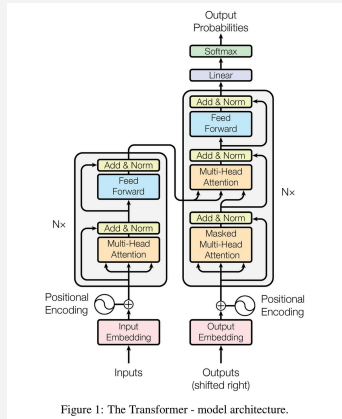


¿Qué necesitamos para hacer un LLM?

Muchos datos



Arquitectura
Transformer

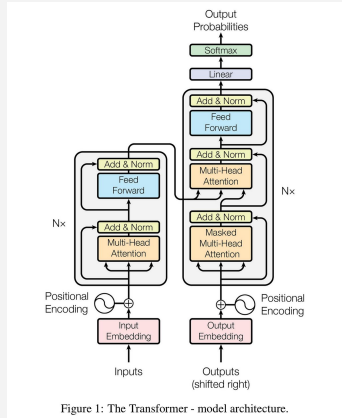


¿Qué necesitamos para hacer un LLM?

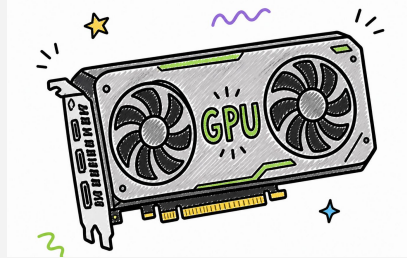
Muchos datos



Arquitectura Transformer



Pre-entrenamiento

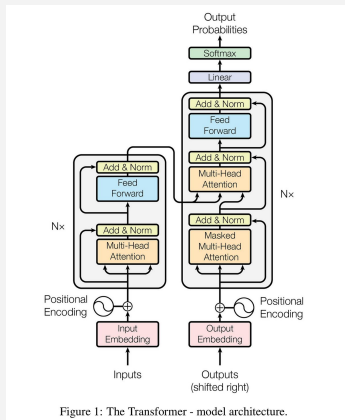


¿Qué necesitamos para hacer un LLM?

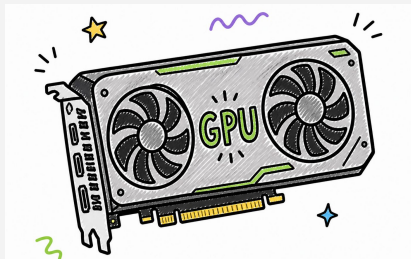
Muchos datos



Arquitectura
Transformer



Pre-entrenamiento



reinforcement
learning de
humanos

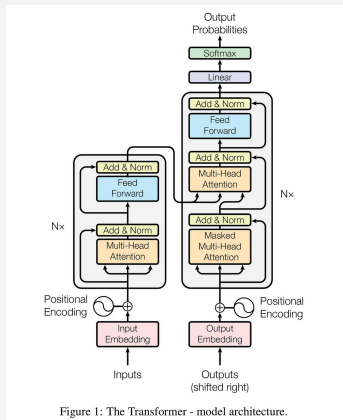


¿Qué necesitamos para hacer un LLM?

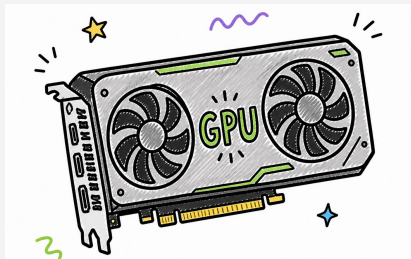
Muchos datos



Arquitectura
Transformer



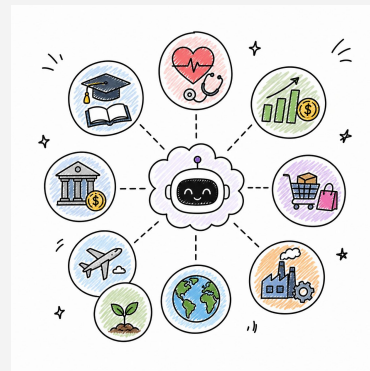
Pre-entrenamiento



reinforcement
learning de
humanos



fine-tuning
(dominio
específico)



Cómo funciona un LLM: Predicción de la siguiente palabra

¿Cuál es la capital de Francia?



Cómo funciona un LLM: Predicción de la siguiente palabra

¿Cuál es la capital de Francia?

¿Cuál es la capital de Francia? La



Cómo funciona un LLM: Predicción de la siguiente palabra

¿Cuál es la capital de Francia?

¿Cuál es la capital de Francia? La



Cómo funciona un LLM: Predicción de la siguiente palabra

¿Cuál es la capital de Francia?
¿Cuál es la capital de Francia? La
¿Cuál es la capital de Francia? La capital



Cómo funciona un LLM: Predicción de la siguiente palabra

¿Cuál es la capital de Francia?
¿Cuál es la capital de Francia? La
¿Cuál es la capital de Francia? La capital
¿Cuál es la capital de Francia? La capital de
¿Cuál es la capital de Francia? La capital de Francia
¿Cuál es la capital de Francia? La capital de Francia es
¿Cuál es la capital de Francia? La capital de Francia es París.
¿Cuál es la capital de Francia? La capital de Francia es París. **<EOS>**



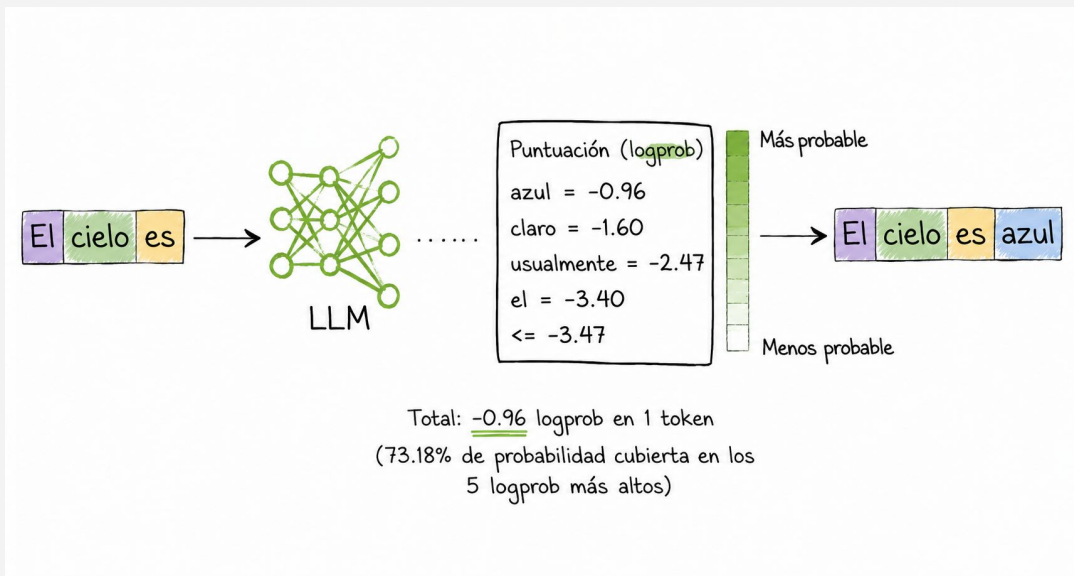
POR QUÉ LOS LLMS ALUCINAN

Y POR QUÉ ES INEVITABLE

¿Por qué los LLMs alucinan?

El modelo no sabe si una afirmación es verdadera

Sabe qué token es estadísticamente más probable según el contexto.



¿Por qué los LLMs alucinan?

El modelo no sabe si una afirmacion es verdadera

Sabe qué token es estadísticamente más probable según el contexto.

cuantas letras r tiene strawberry?

Hay **dos** letras "r" en "strawberry".

La palabra se divide en "straw" y "berry":
una r en "straw" y una r en "berry".

Alucinaciones en el SOC

IOCs fabricados

El modelo puede generar hashes, IPs, dominios que *parecen* reales y son estadísticamente plausibles pero no corresponden a ningún artefacto real.

Puntos ciegos

El modelo no puede razonar sobre información posterior a su entrenamiento. Si le preguntás por contexto ocurrido después de su fecha de corte, va a inventar una respuesta.

Mucha confianza

Una respuesta que suena muy segura puede estar completamente equivocada.

La mitigación correcta no es "usar prompts más específicos": es nunca confiar en afirmaciones factuales del LLM sin verificación independiente contra fuentes reales (VirusTotal, tu SIEM, tu CMDB, inteligencia de amenazas verificada).

Qué no poner en un LLM externo

- Indicadores de compromiso activos de un incidente en curso (IPs, dominios, hashes de una brecha que todavía no fue divulgada)
- Logs con datos de usuarios internos (nombres, emails, actividad de empleados)
- Configuraciones de red, topología de infraestructura, rangos IP internos
- Código propietario de sistemas internos
- Datos de clientes o pacientes (PII, PHI)
- Cualquier información cubierta por NDA o regulaciones como GDPR o HIPAA

La alternativa correcta para entornos con datos sensibles es usar modelos que corren **localmente** (LLaMA, Mistral, Qwen en servidores propios) o proveedores con contratos enterprise que garanticen no usar tus datos para entrenamiento y que cumplan las regulaciones aplicables. El LLM corre en infraestructura propia y nunca sale un paquete a internet con contexto del incidente.

Casos de uso de LLMs

Los LLMs son buenos en tareas donde el texto tiene patrones reconocibles y donde una respuesta aproximada es útil aunque no perfecta y siempre con verificación posterior.

Ejemplos:

- Triage de alertas
- Clasificación de tipo de eventos

```
prompt = f"""
You are a SOC analyst triaging security alerts.
Analyze this alert and provide:
1. Severity: CRITICAL/HIGH/MEDIUM/LOW
2. False positive likely: YES/NO/UNCERTAIN
3. Recommended action: INVESTIGATE_NOW/ESCALATE/MONITOR/CLOSE
4. Reasoning: explain your assessment in 2-3 sentences

Alert data:
{json.dumps(alerta_raw, indent=2)}
"""
```

```
prompt = f"""
Classify this security event into EXACTLY ONE category:
MALWARE_EXECUTION | DATA_EXFILTRATION | PRIVILEGE_ESCALATION |
LATERAL_MOVEMENT | CREDENTIAL_COMPROMISE | PHISHING |
POLICY_VIOLATION | SYSTEM_ERROR | LEGITIMATE_ACTIVITY

Event: {evento_texto}
"""
```

Deobfuscación de código

Si le pasás al LLM un fragmento de código **ofuscado (intencionalmente dificultoso de leer)** y le pedís que lo analice, puede reconocer ciertos patrones y darte una idea general de qué está haciendo.

- sirve para acelerar la primera revisión
- luego decidir si hace falta investigarlo más a fondo en sandbox

Para malware muy sofisticado (exploits de zero-day, técnicas de stealth avanzadas), los LLMs pueden no ser confiables. El malware actual es frecuentemente más complejo que el texto de entrenamiento del LLM.

Un LLM no puede realmente ejecutar el malware para ver qué hace. Solo puede leer código estático. Malware puede tener comportamiento solo en ciertos contextos que el análisis estático no capture.

Los LLMs fueron entrenados hasta cierta fecha. Algo descubierto después de esa fecha, puede no ser reconocido.

Resumen de reportes

Cuando un reporte tiene 50 páginas, **un LLM puede resumirlo en una sola con lo esencial**: familia, vector de ataque, impacto, sistemas afectados, IOCs y acciones recomendadas.

- Se le da el reporte completo y se le pide una salida con estructura fija.

Resumen de reportes

Le pasás un reporte de malware, aunque venga en PDF o texto desordenado, y le pedís que extraiga y clasifique los IOCs.

- Puede identificar IPs, dominios, hashes, rutas de archivo, claves de registro y nombres de procesos, y devolver todo en JSON.
- Algo que antes hacía un analista a mano, ahora se resuelve en segundos y queda listo para usar en threat intel o reglas IDS.

Los LLMs pueden "inventar" IOCs que no existen realmente.

```
prompt = f"""
Extract all IOCs (Indicators of Compromise) from this malware report.

Categorize as:
- IP addresses (with country/ASN if available)
- Domain names
- File hashes (MD5, SHA1, SHA256)
- Registry keys modified
- Files created
- Network ports used
- Process names/behaviors
- URLs

Report:
{reporte_malware}

Format output as structured JSON:
{{
  "ips": [...],
  "dominios": [...],
  "hashes": [...],
  "ports": [...]
}}
```

Generación de reportes de incidentes

1. EXECUTIVE SUMMARY (1 página)
 - Qué sucedió
 - Cuándo sucedió
 - Quién/Qué fue afectado
 - Impacto estimado
 - Recomendaciones
2. TIMELINE (1-2 páginas)
 - Eventos cronológicos desde primer indicador hasta contención
 - Acciones de respuesta tomadas
3. TECHNICAL ANALYSIS (2-5 páginas)
 - Detalles técnicos del incidente
 - IOCs encontrados
 - Métodos de ataque
 - Sistemas comprometidos
4. IMPACT ASSESSMENT (1-2 páginas)
 - Datos potencialmente expuestos
 - Sistemas afectados
 - Duración del acceso no autorizado
5. ROOT CAUSE (1 página)
 - Por qué sucedió el incidente
 - Vulnerabilidades explotadas
6. RECOMMENDATIONS (1-2 páginas)
 - Acciones para prevenir incidentes similares
7. APPENDICES
 - IOCs detallados
 - Logs de referencia
 - Screenshots

- Tenemos la estructura del reporte definida
- Pasarle toda la información que ya recolectó: el usuario afectado, los dispositivos, los IOCs que descubrió, la timeline de eventos, las acciones que tomó.

```
prompt = f"""
Generate a professional incident report based on the following case data.

Case Data:
{json.dumps(datos_caso, indent=2)}

Requirements:
- Write in formal, professional tone
- Include specific dates/times and technical details
- Organize chronologically
- Be comprehensive but concise
- Provide clear recommendations

Generate the full report with all sections:
[EXECUTIVE SUMMARY]
[TIMELINE]
[TECHNICAL ANALYSIS]
[IMPACT ASSESSMENT]
[ROOT CAUSE]
[RECOMMENDATIONS]
"""
```

Generación de reportes de incidentes

- Personalización por tipo de incidente

```
TEMPLATES = {  
    "phishing": {  
        "focus": ["email artifacts", "user actions", "email gateway logs"],  
        "emphasis": "user education"  
    },  
    "malware": {  
        "focus": ["process analysis", "network connections", "file hashes"],  
        "emphasis": "endpoint isolation"  
    },  
    "data_exfiltration": {  
        "focus": ["data accessed", "volume exfiltrated", "destinations"],  
        "emphasis": "scope of exposure"  
    },  
    "insider_threat": {  
        "focus": ["user actions", "data accessed", "timeline"],  
        "emphasis": "behavioral patterns"  
    }  
}
```

```
tipo_incidente = "phishing"  
  
prompt = f"""  
Generate incident report for {tipo_incidente} incident.  
  
Focus particularly on:  
{', '.join(template['focus'])}  
  
Emphasize in recommendations:  
{template['emphasis']}  
  
Case data:  
{json.dumps(datos_caso)}  
"""
```

Generación de reportes de incidentes

Revisión humana y control de calidad del reporte: ¿Incluye todos los IOCs que encontraste? ¿La timeline está en orden? ¿Menciona al usuario afectado? ¿Tiene recomendaciones concretas?

Pero nosotros somos medio vagos: podemos hacer algunos chequeos automáticos!

```
def revisar_reporte(reporte_generado, datos_caso):  
    """Control de calidad del reporte generado"""  
  
    problemas = []  
  
    # Verificación 1: ¿Contiene datos específicos del caso?  
    if not datos_caso['usuario_afectado'] in reporte_generado:  
        problemas.append("Missing affected user name")  
  
    # Verificación 2: ¿Incluye IOCs?  
    for ioc in datos_caso['iocs']['urls']:  
        if ioc not in reporte_generado:  
            problemas.append(f"Missing IOC: {ioc}")  
  
    # Verificación 3: ¿Tiene recomendaciones?  
    if "recommend" not in reporte_generado.lower():  
        problemas.append("Missing recommendations section")  
  
    return {  
        "aprobado": len(problemas) == 0,  
        "problemas": problemas,  
        "requiere_revision": len(problemas) > 0  
    }
```

“

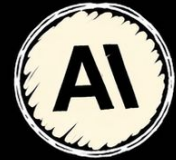
Los LLMs son una herramienta poderosa: aceleran el triage, resumen reportes, generan documentación. Pero el criterio, la verificación y la responsabilidad siguen siendo del analista.

MCP



Model Context Protocol

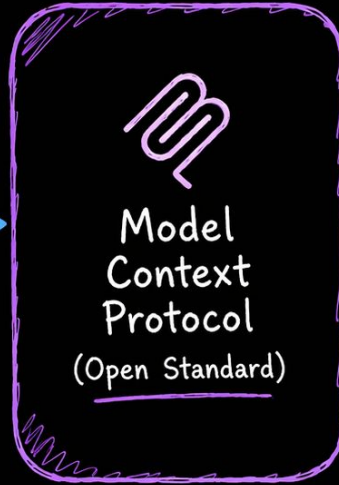
Models / LLMs



Anthropic
Claude



OpenAI



Tools & Services



REST



Apps



Storage

¿Qué problema resuelve MCP?

- Los LLMs solo pueden procesar texto
- Para actuar necesitan conectarse a herramientas externas
- Sin un estándar, cada integración es ad-hoc
- MCP es ese estándar

Arquitectura de MCP

- Tools: el LLM ejecuta acciones
- Resources: el LLM lee datos
- Prompts: plantillas reutilizables

```
@mcp.resource(
    "soc://lab/blacklist",
    name="lab_blacklist",
    description="Blacklist semilla del laboratorio (solo lectura).",
    mime_type="text/plain",
)
def lab_blacklist() -> str:
```

```
@mcp.prompt(
    name="investigate_ssh_alert",
    description="Plantilla para investigar una alerta SSH / brute force.",
)
def investigate_ssh_alert(src_ip: str, user: str) -> str:
    """Prompt parametrizado; el cliente MCP lo pide con get_prompt()."""
    return (
        f"Investigá posible brute force SSH desde {src_ip} contra el usuario '{user}'. "
        "Usá el contexto PRE-IA del ticket y tools de historial/ticket si hace falta. "
        "No ejecute bloqueos; devolvé un análisis breve en español y next steps."
    )
```

server.py Ln 41-42

```
@mcp.tool()
def check_ip_reputation(ip: str) -> str:
```

server.py Ln 47-48

```
@mcp.tool()
def get_user_login_history(user: str) -> str:
```

server.py Ln 56-57

```
@mcp.tool()
def geolocate_ip(ip: str) -> str:
```

server.py Ln 62-63

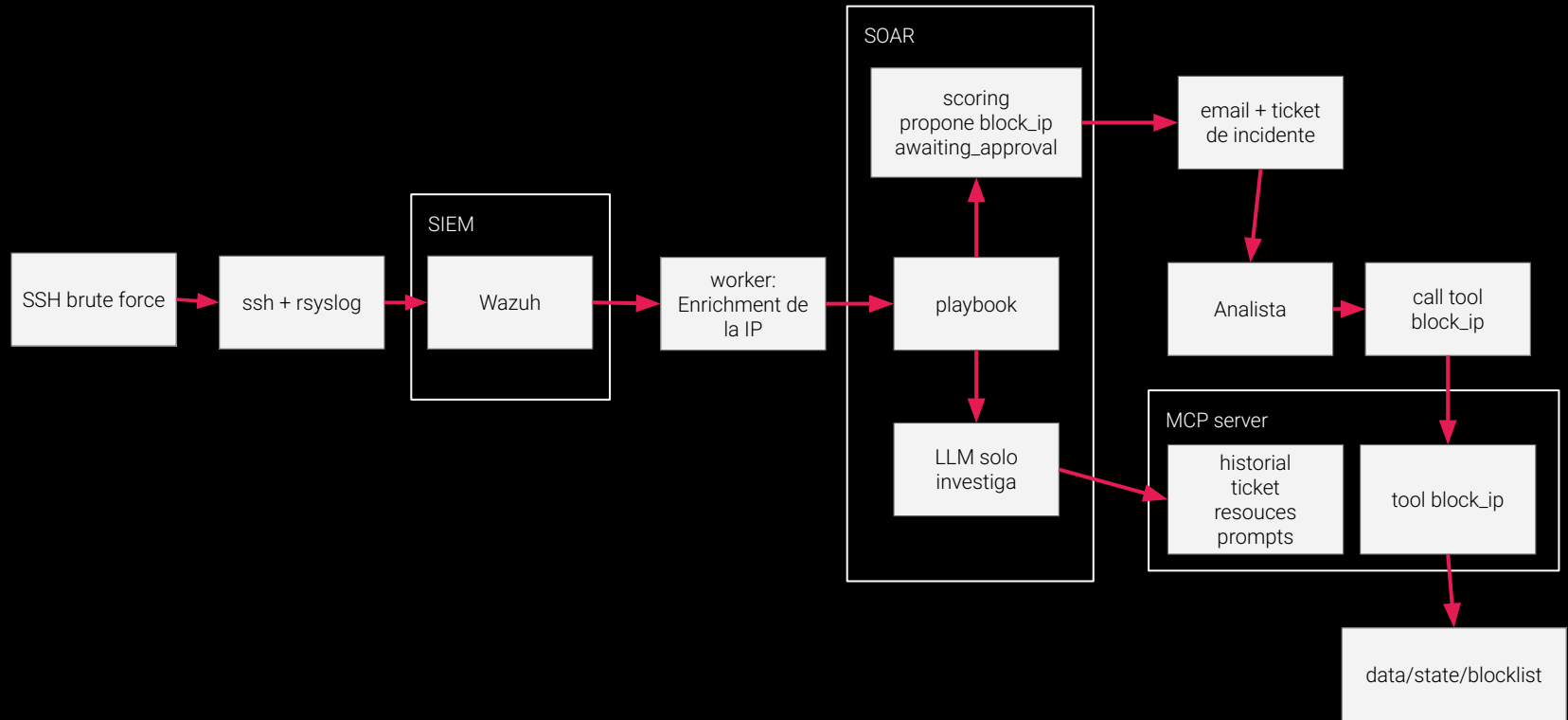
```
@mcp.tool()
def create_incident_ticket(title: str, severity: str, summary:
```

server.py Ln 80-81

```
@mcp.tool()
def block_ip(ip: str, approved: bool = False) -> str:
```

“

El MCP no le da inteligencia al LLM, le da manos.



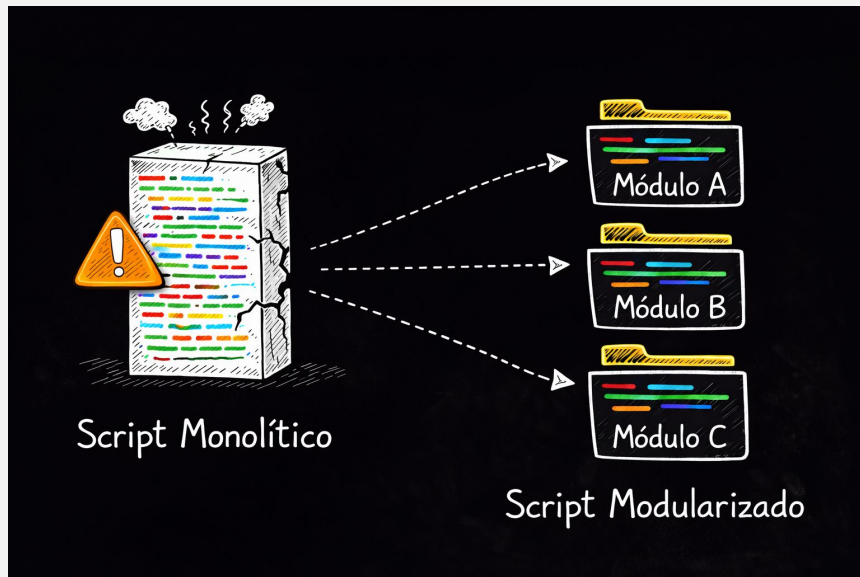
UN REPASO DE LO VISTO

Automatización con Python

- Scripting en Python para tareas de seguridad
- Automatización de tareas repetitivas
- Buenas prácticas: modularidad, logging, testing básico



Integraciones y Conectores



- Desarrollo de conectores reutilizables
- Patrón ConectorBase
 - un fix beneficia a todos los conectores
- Patrón Registry
 - el código no sabe cómo se inicializó nada, solo pide lo que necesita
- Integración con APIs externas

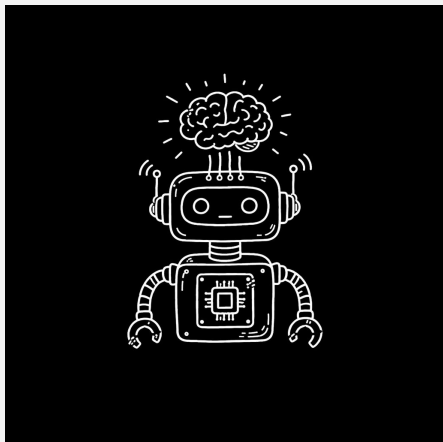
SOAR y automatización de respuestas

- Runbooks y playbooks
- Orquestación de herramientas de seguridad
- Automatización de respuesta a incidentes

ML en Seguridad y Detección de Anomalías

- Algoritmos comunes: árboles de decisión, Random Forest, Isolation Forest, SVM, redes neuronales, K-Means, DBSCAN, autoencoders
- Anomaly Detection: detectar lo que se desvía de lo normal
- UBA (User Behavior Analytics)
- UEBA: extiende UBA a entidades (servidores, routers, apps, IoT)
- UEBA + SIEM: reducir ruido, priorizar investigaciones

Introducción a IA y Machine Learning



- Qué es la IA y el ML; diferencia con reglas estáticas
- Aprendizaje supervisado vs. no supervisado
- Workflow: recolección → features → entrenamiento → evaluación → deploy
- Métricas: accuracy, precision, recall, F1
- Ajuste continuo del modelo (re-entrenamiento con feedback de analistas)
- Limitaciones del ML en seguridad: cold start, ataques adversariales, concept drift, garbage in/garbage out

NLP y LLMs en Seguridad

- NLP en seguridad: clasificar alertas, extraer IOCs, detectar patrones en texto
- Cómo funciona un LLM
- Alucinaciones: el modelo no valida hechos, predice tokens probables
- Riesgos: IOCs fabricados, puntos ciegos, exceso de confianza
- Qué no enviar a un LLM externo: IOCs activos, logs con PII, topología de red, código propietario
- Casos de uso: triage de alertas, clasificación de eventos, deobfuscación de código, resumen de reportes, generación de reportes de incidentes con estructura fija

Model Context Protocol (MCP)

- Problema que resuelve: los LLMs solo procesan texto; MCP es el estándar para conectarlos a herramientas externas
- Arquitectura: Tools (el LLM ejecuta acciones), Resources (el LLM lee datos), Prompts (plantillas reutilizables)
- Casos de uso en seguridad: consultar reputación de IOCs en VirusTotal, buscar eventos en el SIEM, crear tickets de incidentes, aislar hosts en el EDR

Muchas gracias!

PREGUNTAS?

Me pueden encontrar en

Mail: flor.p.vela@gmail.com

Github: [@florvela](#)

